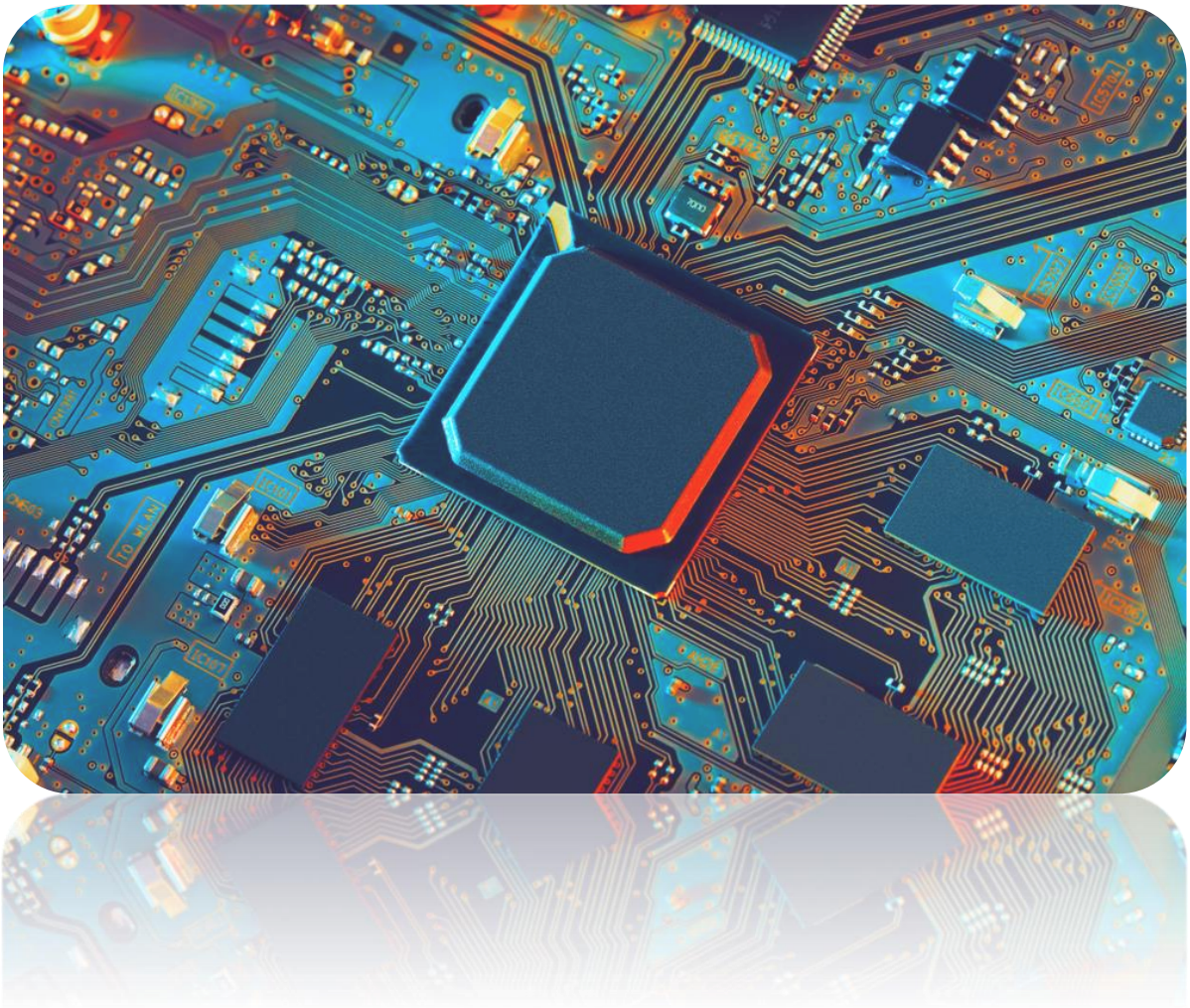
CE-222L
Computer Organization and Assembly Language



Faculty of Computer Science and Engineering
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Table of Contents

Lab 1: Introduction to Verilog 4

Objectives: 4

Verilog: 4

Verilog module: 4

Dataflow Modeling: 5

Xilinx ISE: 7

Starting a New Project 8

Example of NAND gate in Verilog: 13

Lab Tasks 16

Lab 2: Verilog Gate Level Modeling 17

Objectives: 17

Introduction: 17

Gate Types: 17

Examples: 19

Lab Tasks: 25

Lab 3: Verilog Behavioral Modeling 26

Objectives: 26

Introduction: 26

Structured Procedures: 26

Conditional Statements: 28

Examples: 30

Lab Tasks: 42

Lab 4: Sequential Circuits in Verilog HDL 43

Objectives: 43

Introduction: 43

Types of Logic Circuits: 43

Combinational Circuits: 43

Sequential circuits: 44

Examples: 44

Lab Tasks: 54

Lab 5: Memories in Verilog 55

Lab Objectives: 55

Arrays declaration in Verilog: 55

Concatenation in Verilog 57
Memories: 58
Lab Tasks: 63
Lab 6: Register File in Verilog 64
Lab Objectives: 64
Register file: 64
Functionality: 65
Example: Implementation of RAM: 67
Lab Tasks: 71
Common Errors / Instructions: 72
Lab 7: Data Path, Control Units, and Immediate Generation 73
Lab Objectives: 73
Data Path: 73
Example: 74
Immediate Generation: 78
Control Unit: 80
ALU Control Unit: 82
Lab Tasks: 83
Lab 8-11: Implementation of RISC-V Processor in Verilog 85
Lab Objectives: 85
Introduction: 85
History of RISC-V: 85
Key Features of RISC-V: 85
Advantages of RISC-V: 85
Main Components: 86
Lab Task: 90

Lab 1: Introduction to Verilog

Objectives:

- Understand the fundamental concepts of Verilog Hardware Description Language (HDL).
- Learn the syntax and structure of Verilog code for designing digital circuits.
- Learn how to use Verilog for simple arithmetic and data manipulation tasks.
- Implement basic logic gates using Verilog Dataflow Modeling and simulate their behavior.

Verilog: Verilog is the combination of the terms *"Verification"* and *"Logic"*. It is hardware description language or a special type of programming language which describes the hardware implementations of digital systems and circuits. It is most commonly used in the design and verification of digital circuits. It is also used in the verification of analog circuits and mixed-signal circuits.

Verilog module:

Verilog provides the concept of a module. A module is the basic building block in Verilog. A module can be an element or a collection of lower-level design blocks.
In Verilog, a module is declared by the keyword module. A corresponding keyword endmodule must appear at the end of the module definition. Each module must have a *module_name*, which is the identifier for the module, and a *module_terminal_list*, which describes the input and output terminals of the module.

module *<module_name>* **(***<module_terminal_list>***);** //outputs first then inputs

...

<module internals> // Also called module body

...

endmodule

- Verilog is case sensitive.
- Reserved keywords are written in lower cases.
- A semicolon is used to terminate the statement.
- The comment rule is same as C Programming Language.

Purpose of a module: A module represents a design unit that implements certain behavioral characteristics and well get converted into a digital circuit during synthesis. Any combination of the inputs can be given to the module, and it will provide a corresponding output. This allows the same module to be reused to form bigger modules that implement more complex hardware.

- Single line comment starts with "//".
  For example – *//Example of a Verilog single line comment*

- Multiline comments start with – ‘ /*’ and ends with ‘*/’.
  For example –

  \/ Example*
  Of Verilog multiple
  Line comment/*

Verilog Data Types: The primary intent of data types in the Verilog language is to represent data storage elements like bits in a flipflop and transmission elements like wires in that connect logic gates and sequential circuits.

Verilog Operators: Verilog has three fundamental operators for Verilog HDL. They are given below.

Unary Verilog operators: These types of Verilog operators come first of the operands.

For example: `x = ~ y;  // Here '~' is a unary operator`

Binary Verilog operators: These types of Verilog operators come in between two operands.

For example: `x = y || z; Here '||' is a binary operator.`

Ternary Verilog operators: These types of Verilog operators use two different operators to differentiate three operators.

For example: `x = y?  z: w; here '?' and ':' are ternary operators.`

Verilog HDL’s categorical operators are – arithmetical, logical, relational, bitwise, shift, concatenation, and equality. Different types of Verilog operators and their symbols are given in the below table.

Dataflow Modeling:

Dataflow modeling provides the means of describing combinational circuits by their function rather than by their gate structure. Dataflow modeling uses several operators that act on operands to produce the desired results. Verilog HDL provides about 30 operator types.

Type of Operator	Symbol	Operation	Operands needed
Arithmetic	*	*Multiplication*	*Two*
	/	*Division*	*Two*
	+	*Addition*	*Two*
	–	*Subtraction*	*Two*
	%	*Modulus*	*Two*
Logical	!	*Negation*	*One*

	&&	*AND*	*Two*
	||	*OR*	*Two*
Relational	>	*Greater than*	*Two*
	<	*Less than*	*Two*
	> =	*Greater than or equal to*	*Two*
	< =	*Less than or equal to*	*Two*
Equality	==	*Equals to*	*Two*
	! =	*Not equals to*	*Two*
	===	*Case equal*	*Two*
	! ==	*Case not equal*	*Two*
Bitwise	~	*Negation*	*One*
	&	*Bitwise AND*	*Two*
	|	*Bitwise OR*	*Two*
	^	*Bitwise XOR*	*Two*
	~^	*Bitwise XNOR*	*Two*
Reduction	&	*Reduction AND*	*One*
	~&	*Reduction NAND*	*One*
	|	*Reduction OR*	*One*
	~|	*Reduction NOR*	*One*
	^	*Reduction XOR*	*One*
	^~	*Reduction XNOR*	*One*
Shift	>>	*Right Shift*	*Two*
	<<	*Left Shift*	*Two*

Concatenation	*{ }*	*Concatenation*	*Can be of any numbers*
Replication	*{ { } }*	*Replication*	*Can be of any numbers*
Conditional	*? :*	*Conditional*	*Three*

Verilog Number Specifications
Verilog numbers are of two types, sized numbers and unsized numbers.

Sized Verilog numbers: The general structure for representing sized numbers in Verilog HDL is given below.
<size>'<base format><numbers>
For example – *4'b1011*.

- ***size:*** Size is the number of digits the main number has. Size is described using decimal values.
- ***base format:*** Base format suggests which type of number it would be. There are several types – binary (given by – 'b'), decimal (given by – 'd'), octal (given by – 'o'), hexadecimal (given by – 'h'). If there is no specification for base format, then by default it is a decimal number.
- ***numbers:*** The main number you want to put in.

Unsized Verilog numbers: These numbers do not require any specified size.
The general structure for representing unsized numbers in Verilog HDL is given below.
'<base format><numbers>
For example – *'h3456.*
This is an unsized Verilog number which describes that it is a hexadecimal number.

Negative Numbers: If you want to declare a number as a negative number, then put a minus symbol (-) before the number.
For example: – *345; it is a negative, unsized, decimal number.*

Xilinx ISE:

Xilinx ISE (Integrated Software Environment) is a comprehensive suite of software tools developed by Xilinx for designing, implementing, and analyzing digital systems on Xilinx FPGAs and CPLDs (Complex Programmable Logic Devices). Originally introduced in 1996, Xilinx ISE has evolved into a versatile and powerful platform, offering a seamless integration of design entry, synthesis, implementation, and verification processes. It supports a wide range of hardware description languages, including VHDL and Verilog, facilitating the creation of complex digital circuits and systems. Xilinx ISE plays a pivotal role in the field of computer organization and assembly language by providing students and professionals with a robust environment to explore and implement digital designs. In the lab manual, inclusion of Xilinx ISE emphasizes the practical application of theoretical concepts, enabling learners to gain hands-on experience in the design and implementation of digital systems using industry-standard tools. This exposure to Xilinx ISE contributes to a holistic understanding of computer organization and assembly language concepts within the context of contemporary FPGA technology.

Starting a New Project

1. To create a new project, open Project Navigator either from the Desktop icon or by selecting:

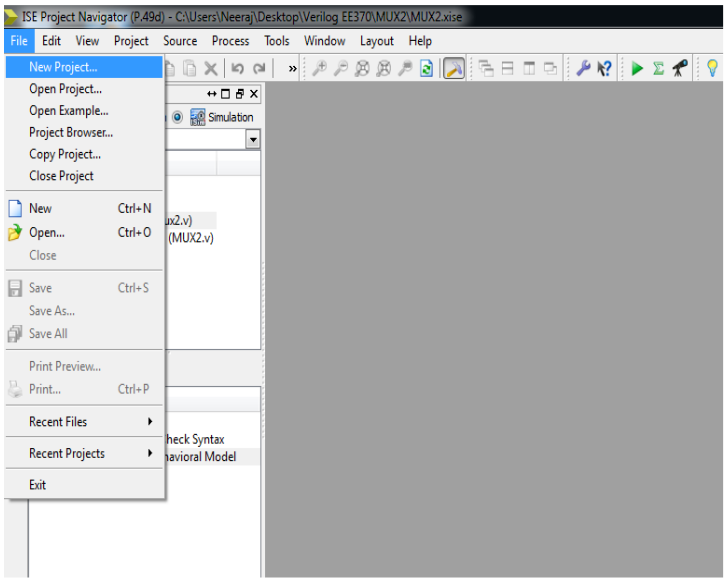
a) Start > Programs > Xilinx ISE Design Suite 14 > ISE > Project Navigator. In Project Navigator.

b) Select the New Project option from the Getting Started menu (or by selecting Select File > New Project).

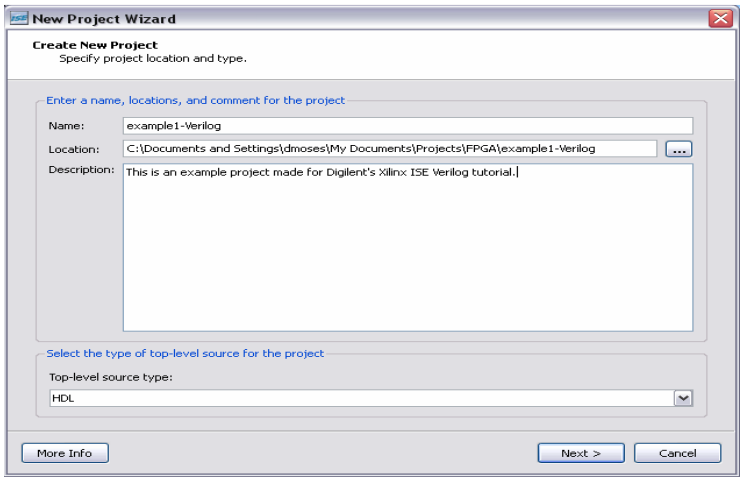
c) This brings up a Dialog box where you can enter the desired project name and project location.

d) Choose a meaningful name for easy reference. Here we call this project "example1-Verilog" and save it in a local directory.

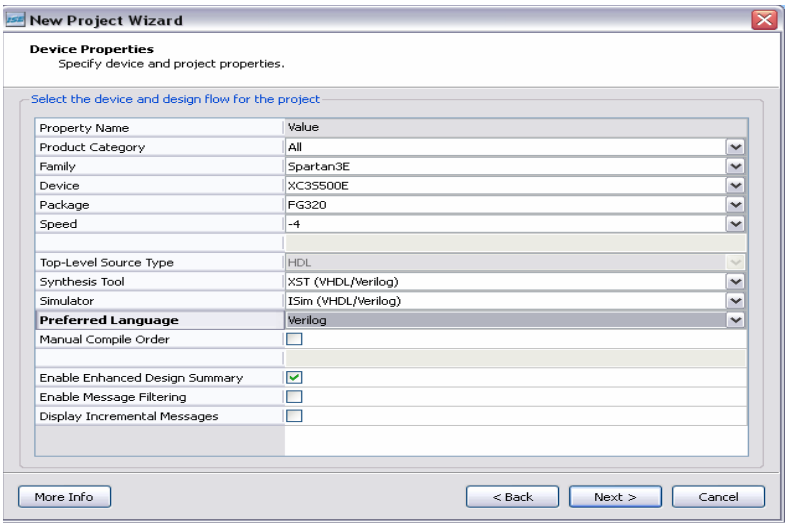
Open New Project



Choose the location to create a New Project. Always choose something other than User folder as it is read only. The best way is to create a folder on desktop, name it and copy its *url* and paste it in the "location" of the "new project wizard".

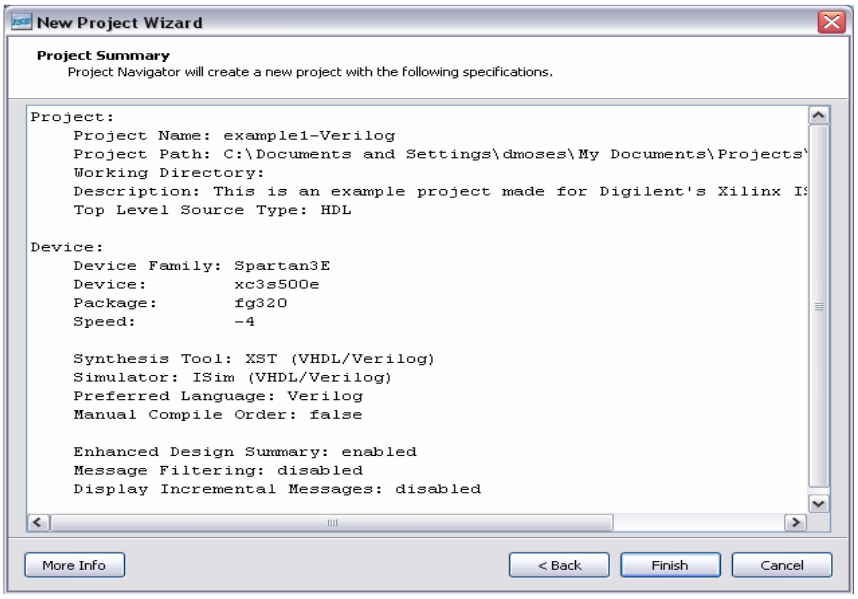


2. (The next step is to select the proper Family, Device, and Package for your project. This depends on the chip you are targeting for this project. The appropriate settings for a project suited for the Nexys2 500k board are as follows :



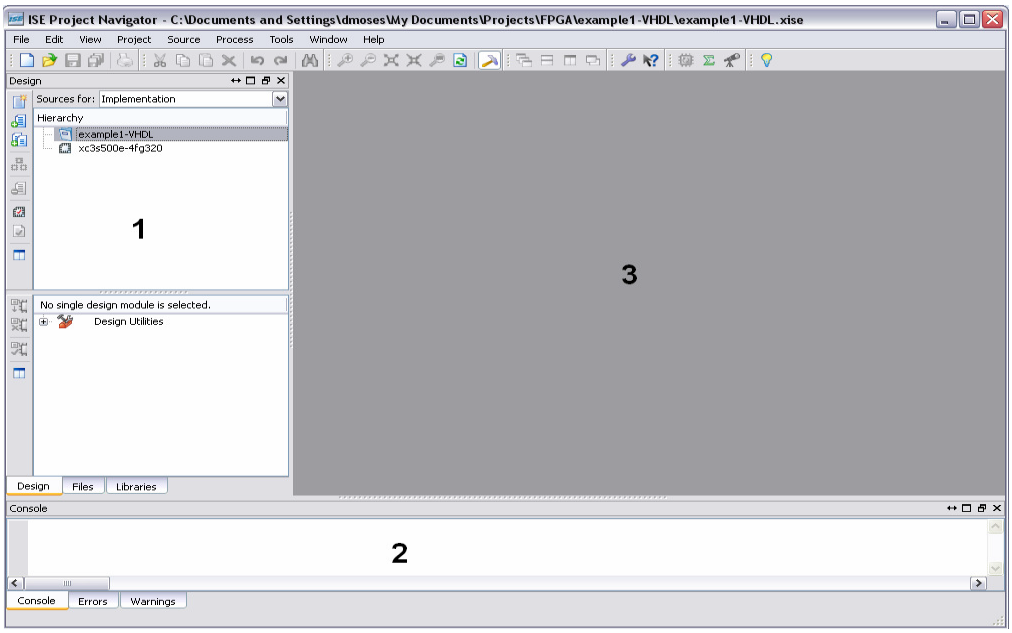
Since this lab does not require physically burn the logic on a chip so we will leave this window as it is. Just click Next.

3. Once the appropriate settings have been entered, click *Next*. The next two dialog boxes give you the option of adding new or existing source files to your project. Since we will fulfill these steps later, click *Next* without adding any source files.



Project Navigator Overview

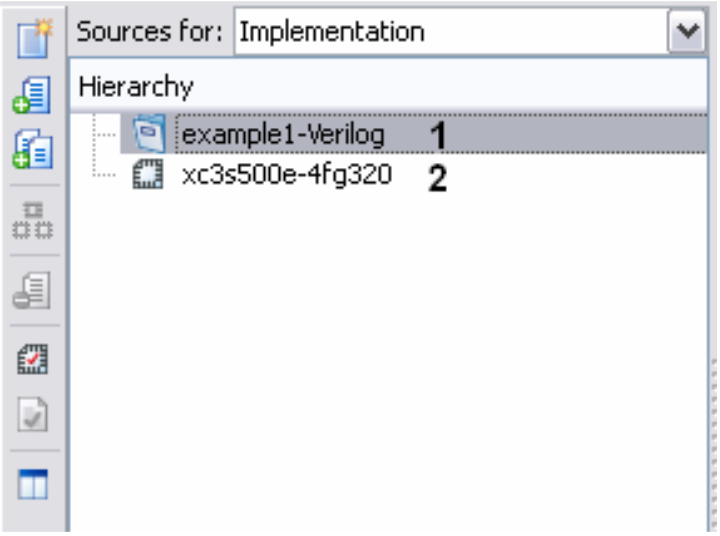
Once the new project has been created, ISE opens the project in Project Navigator. Click the Design tab to show the Design panel and click the Console tab to show the Console panel.



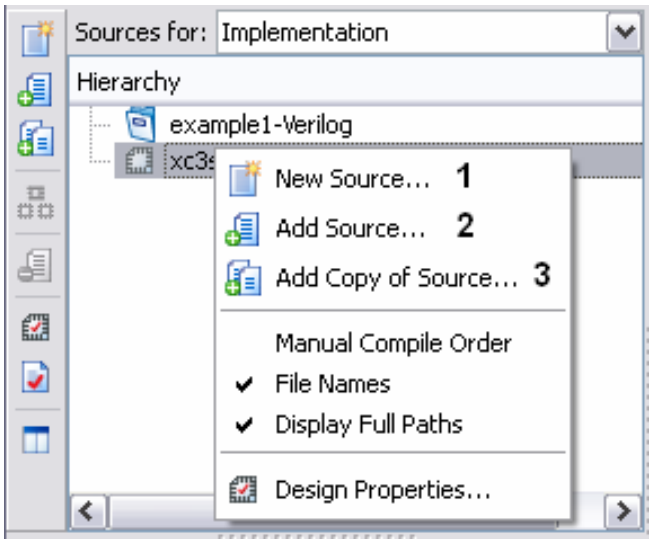
- The Design panel (1) contains two windows: a Sources window that displays all source files associated with the current design and a Process window that displays all available processes that can be run on a selected source file.
- The Console panel (2) displays status messages including error and warning messages.
- The HDL editor window (3) displays source code from files selected in the Design panel.

Adding New Source Files

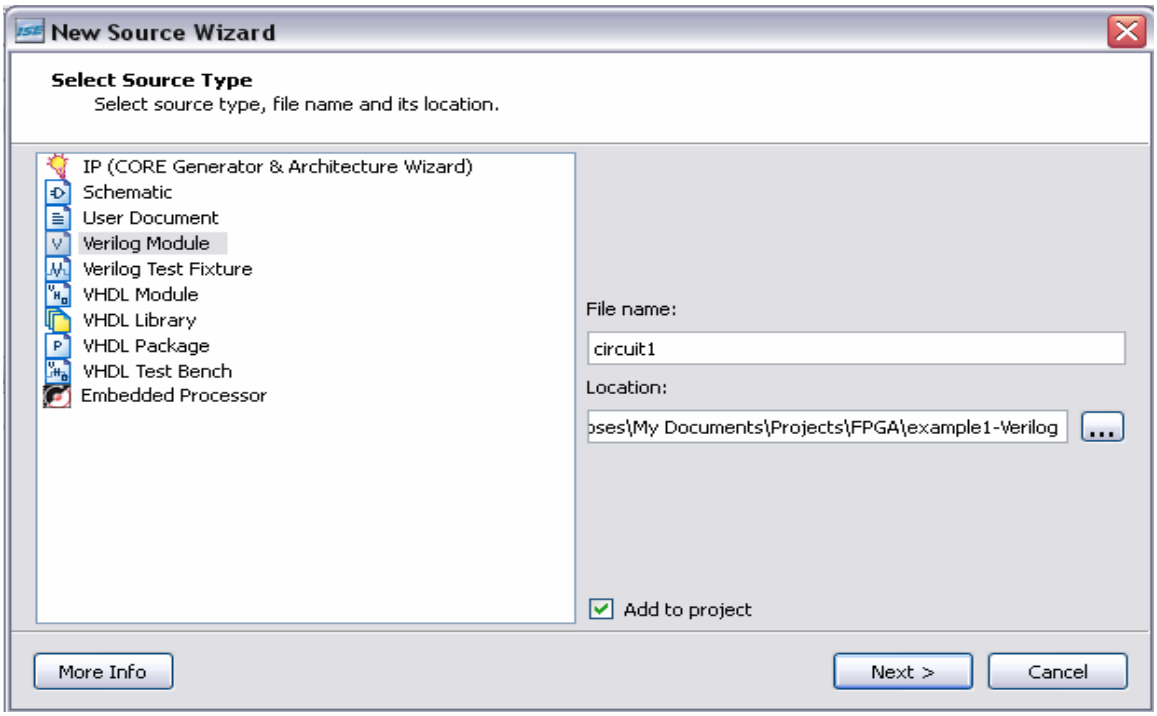
Once the new project is created, two sources are listed under sources in the Design panel: the Project file name and the Device targeted for design.



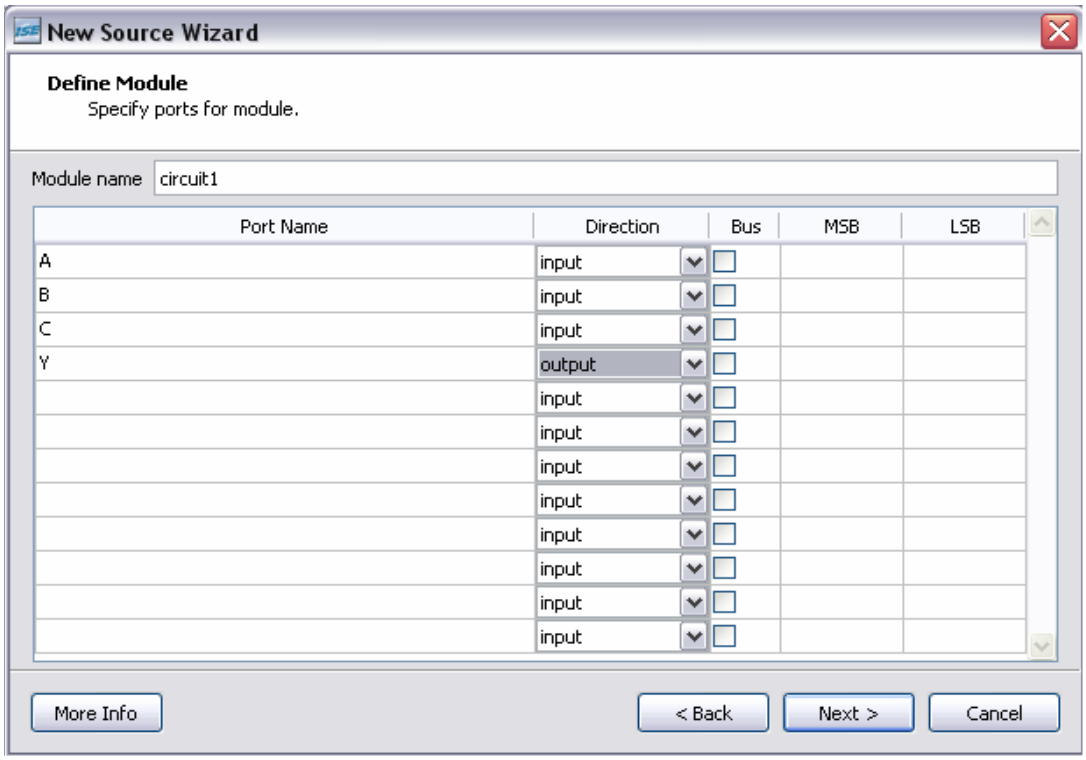
You can add a new or existing source file to the project. To do this, right-click the target device and select one of the three options for adding source files.



We create a new source file, so select New Source from the list. This starts the New Source Wizard, which prompts you for the Source type and file name. Select Verilog Module and give it a meaningful name e.g. circuit1



When you click Next, you have the option of defining top-level ports for the new Verilog module. We chose A, B, and C as input ports and Y as an output port.



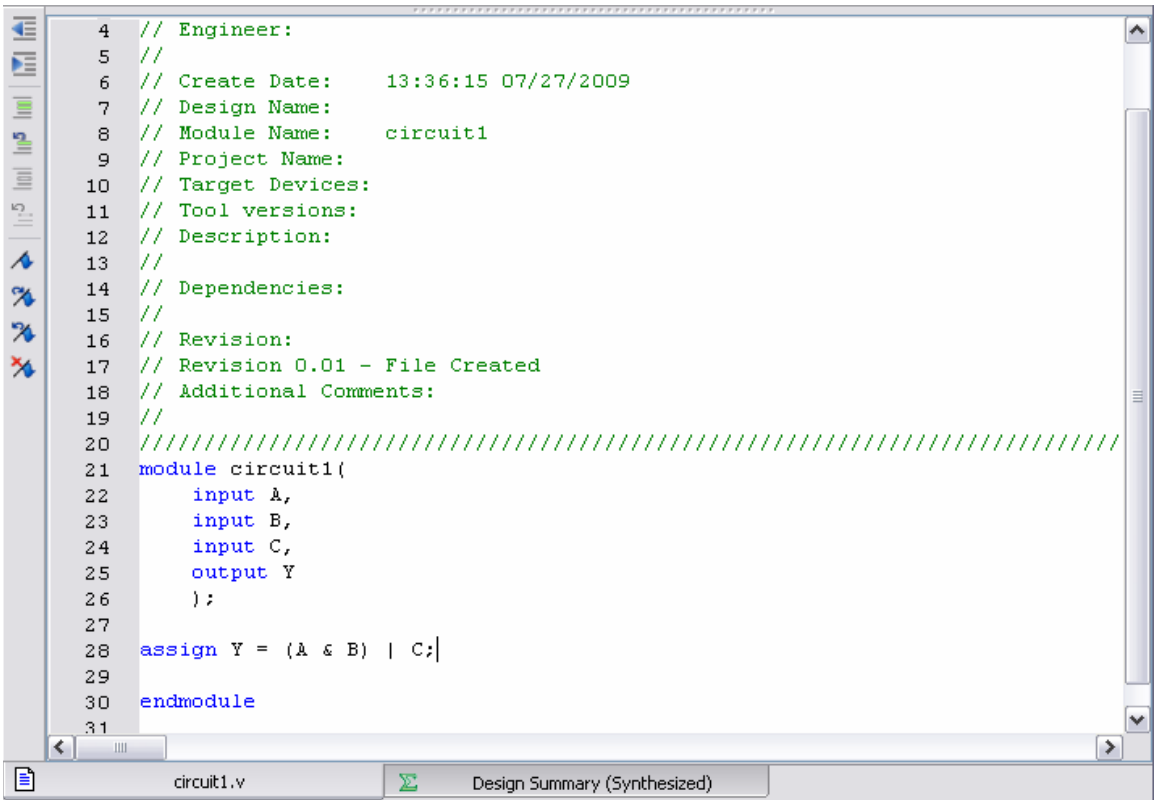
HDL Editor Window

Once you have created the new Verilog file, the HDL Editor Window displays the circuit1.v source code.

The Xilinx tools automatically generate lines of code in the file to get you started with circuit development. This generated code includes:

1. a module statement
2. a comment block template for documentation

The actual behavioral or structural description of the given circuit is to be placed between the "module" and "endmodule" statements in the file. Between these statements, you can define any Verilog circuit you wish. In this tutorial, we use a simple combinational logic example, and then show how it can be used as a structural component in another Verilog module. We start with the basic logic equation: $Y = (A \cdot B) + C$.



```
// Engineer:
//
// Create Date:    13:36:15 07/27/2009
// Design Name:
// Module Name:    circuit1
// Project Name:
// Target Devices:
// Tool versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////
module circuit1(
    input A,
    input B,
    input C,
    output Y
    );

assign Y = (A & B) | C;

endmodule
```

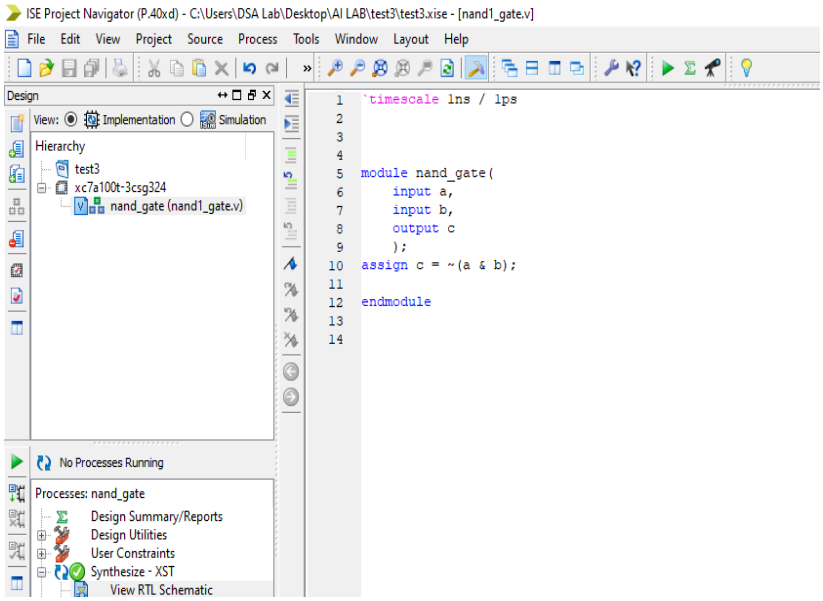
circuit1.v | Design Summary (Synthesized)

If there are any syntax errors in the source file, ISE can help you find them. For example, the parentheses around the A and B inputs are crucial to this implementation; without them, ISE would return an error during synthesis info.

Example of NAND gate in Verilog:

Code

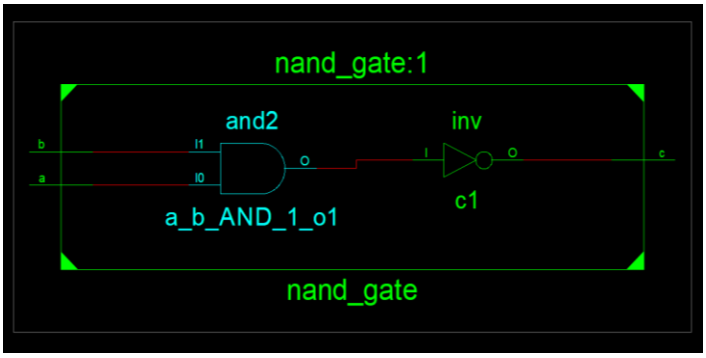
```
module nand_gate(
   input a,
   input b,
   output c
   );
assign c = ~(a & b);
endmodule
```



RTL Schematics and Syntax Checking

The RTL schematics can be viewed by double clicking on View RTL Schematic in synthesize-XST, the example used here is that of a Nand gate.

RTL



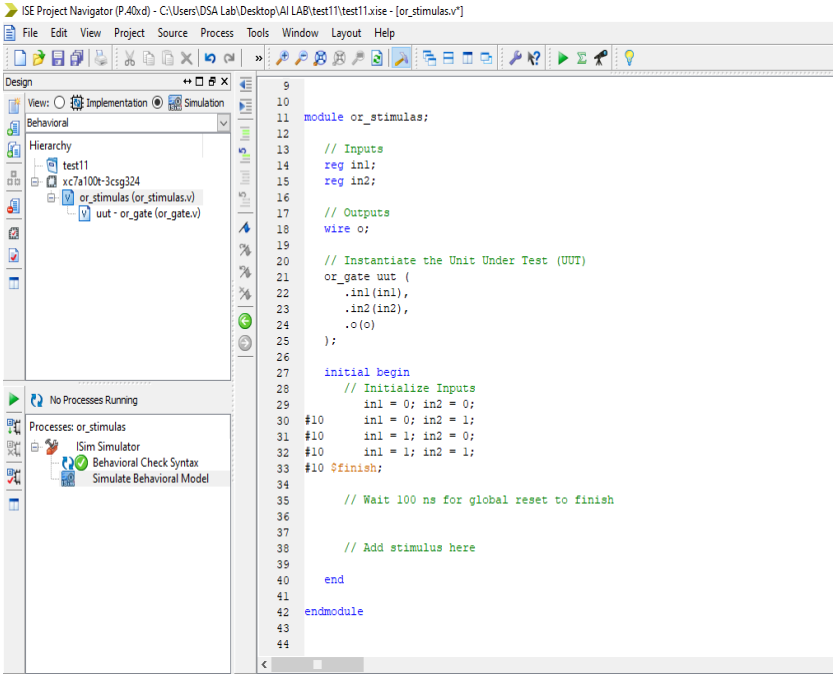
Stimulus/Testbench:

A test bench is a simulation environment crucial in digital system design. Comprising simulation models and stimuli, it allows designers to assess the functionality and performance of circuits before actual implementation. In the realm of hardware description languages like VHDL or Verilog, test benches play a vital role in identifying and rectifying design errors early in the development cycle. By subjecting circuits to various test scenarios, input conditions, and corner cases, test benches ensure thorough validation, reducing the likelihood of errors and enhancing overall design quality.

To create a Test bench, create New Source. Select Verilog Test Fixture. Template of Test bench will be created instantiating the Nand gate module.

```
module nand1_stimulas;
reg a;
reg b;
wire c;

nand_gate uut (
                .a(a),
                .b(b),
                .c(c)
            );
initial begin
a = 0; b = 0;
#10   a = 0; b = 1;
#10   a = 1; b = 0;
#10   a = 1; b = 1;
#10 $finish;
end
endmodule
```

Initial block: Initial blocks cause particular instructions to be performed at the beginning of the simulation before any other instructions operate. Initial blocks only operate once.

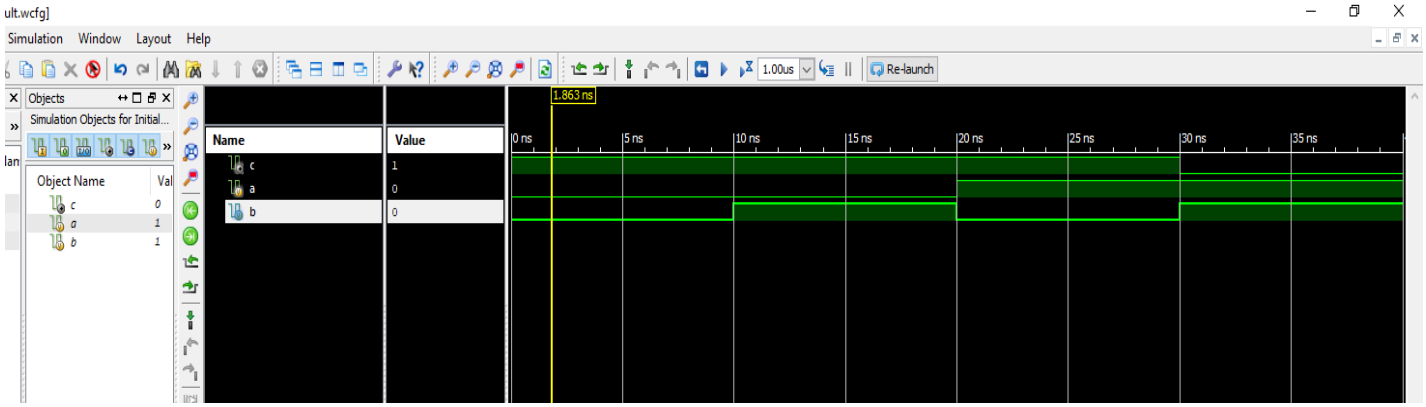


Double click on Simulate Behavioral Model option.

OUTPUT

This is the simulation window. You can verify the working using waveforms.

When both the inputs a & b are 1, the output is 0 as shown below in the waveform and it's 1 for all other input combinations. This verifies our NAND Gate implementation in Verilog.



Lab Tasks

Implement the following logic gates in Xilinx ISE and implement test benches to verify their working. Visualize their behavior in waveforms.

A)

1. AND
2. OR
3. XOR
4. NOT

B) F = (AB) + (CD)

C) Implement 4 inputs AND Gate. It should take 4 1-bit inputs (a,b,c,d) and output should be 1 only when all inputs are 1.

Home Task

Install Xilinx on your PC/Laptop. You can get it from Lab Attendant of your respective lab. (System Architecture / Digital System Lab)

Lab 2: Verilog Gate Level Modeling

Objectives:

- Learn the fundamental logic gates and their implementation using Gate Level Modeling
- Understand and implement combinational logic circuits by utilizing Verilog constructs for gate-level modeling.
- Develop Verilog testbenches to simulate and verify the functionality of gate-level models.
- Interpret timing diagrams generated during simulation to understand the dynamic behavior of digital circuits.

Introduction:

In Verilog, most of the digital designs are done at a higher level of abstraction like RTL. However, it becomes natural to build smaller deterministic circuits at a lower level by using combinational elements such as AND and OR. Modeling done at this level is called gate-level modeling as it involves gates and has a one-to-one relationship between a hardware schematic and the Verilog code. Verilog supports a few basic logic gates known as primitives, as they can be instantiated, such as modules, and they are already predefined.

Gate level modeling is virtually the lowest level of abstraction because switch-level abstraction is rarely used. Gate level modeling is used to implement the lowest-level modules in a design, such as multiplexers, full-adder, etc. Verilog has gate primitives for all basic gates.

Gate Types:

A logic circuit can be designed using logic gates. Verilog supports basic logic gates as predefined primitives. These primitives are instantiated like modules except that they are predefined in Verilog and do not need a module definition. All logic circuits can be designed by using basic gates. There are two classes of basic gates: and/or gates and buf /not gates.

1. And/ Or Gates

And/or gates have one scalar output and multiple scalar inputs. ***And, or, xor, nand, nor, xnor*** **.**

2. Buf /Not Gates

Buf/not gates have one scalar input and one or more scalar outputs. ***Buf, not***

Below illustrates the utilization of these gates in Verilog Gate Level Modeling:

```
// AND Gate
and (output, input1, input2);

// OR Gate
or (output, input1, input2);
```

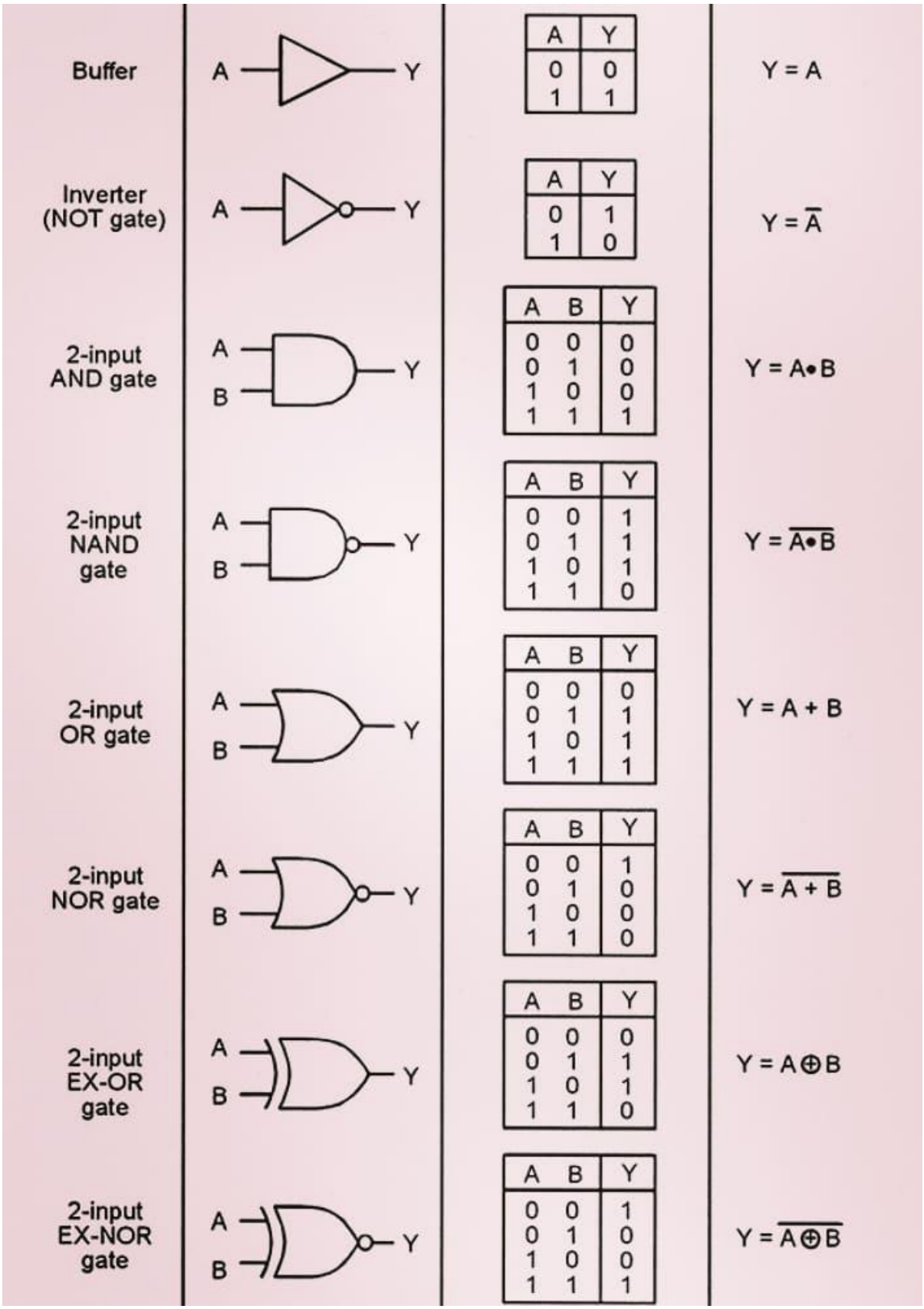


Fig. 1. Basic Logic gates

Syntax

```
module and (z,x,y);
input x,y;
output z;
and a1 (z,x,y);
endmodule
```

Examples:

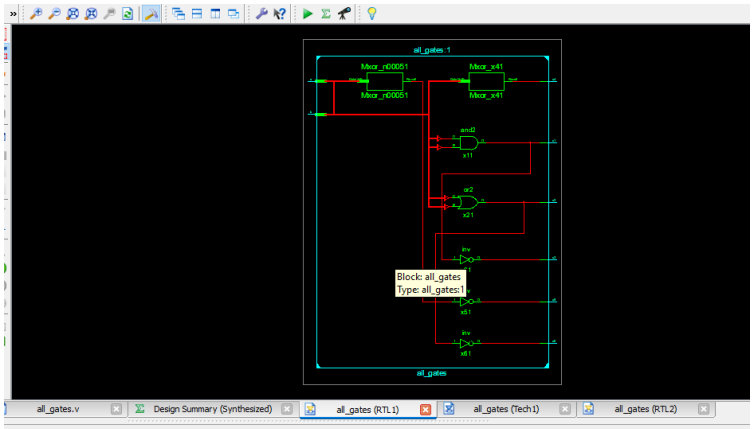
<u>Example # 1 Implementation of all basic gates:</u>

Implement all basic gates (Gate level designing) in Xilinx. implement test benches to verify their working. Visualize their behavior in waveforms.

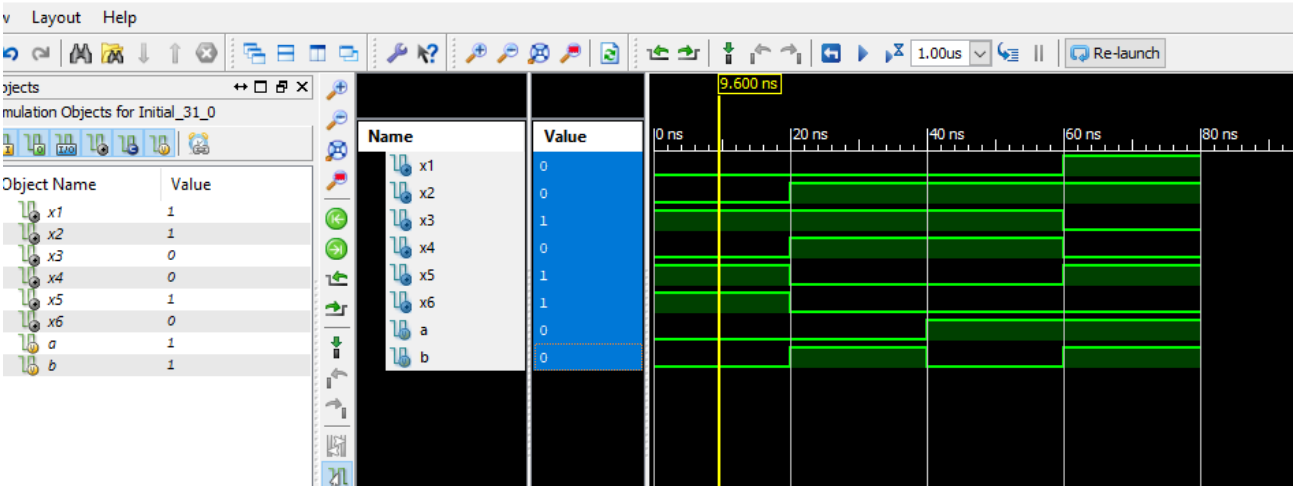
Code

```
module all_gates(
   input a,
   input b,
   output x1,
   output x2,
   output x3,
   output x4,
   output x5,
   output x6
   );
and (x1,a,b);
or  (x2,a,b);
nand (x3,a,b);
xor (x4,a,b);
xnor (x5,a,b);
nor (x6,a,b);
endmodule
```

RTL



Waveform

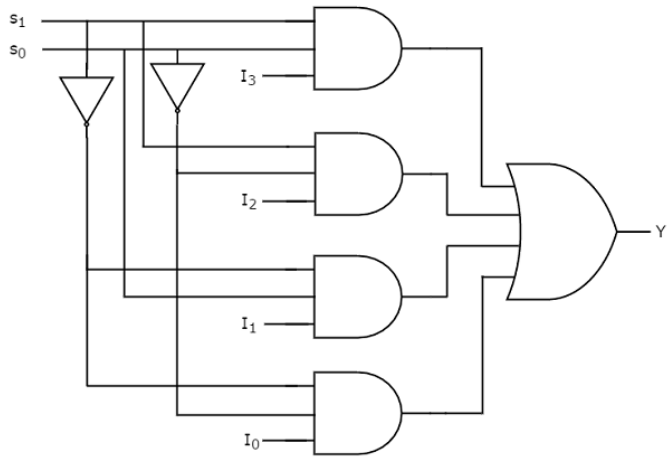


You can verify any gate by checking output waveform against any combination of input.
For example, x3 and x6 are outputs of NAND and NOR gate, which are high when inputs (a & b) are low.

Wire: *In Verilog, wire is a keyword used to declare a net type. In the context of digital design and hardware description languages (HDLs) like Verilog, a wire is used to represent a connection between different elements in the circuit. It is often used to connect the outputs of one module or gate to the inputs of another.*

Example # 2

Implement **4x1** MUX (gate level designing) in Xilinx. implement test benches to verify their working. Visualize their behavior in waveforms.



Code:

```
module mux_4x1(op,i0,i1,i2,i3,s0,s1);
input i0,i1,i2,i3;
input s1,s0;
```

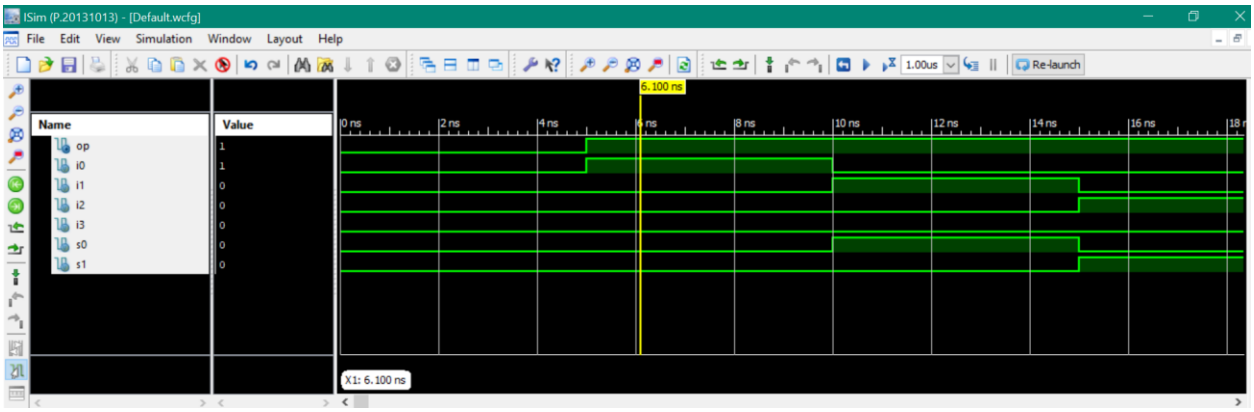
```
output op;
wire s1bar,s0bar;
wire w1,w2,w3,w4;

not n1(s1bar,s1);
not n2 (s0bar,s0);
and a1(w1,i0,s1bar,s0bar);
and a2 (w2,i1,s1bar,s0);
and a3(w3,i2,s1,s0bar);
and a4(w4,i3,s1,s0);
or o1 (op,w1,w2,w3,w4);
endmodule
```

Test Bench:

```
`timescale 1ns / 1ps
module mux_4x1_test;
        reg i0;
        reg i1;
        reg i2;
        reg i3;
        reg s0;
        reg s1;

        // Outputs
        wire op;

        // Instantiate the Unit Under Test (UUT)
        mux_4x1 uut (
                .op (op),
                .i0 (i0),
                .i1 (i1),
                .i2 (i2),
                .i3 (i3),
                .s0 (s0),
                .s1 (s1)
        );

        initial begin
                // Initialize Inputs
                        i0 = 0; i1 = 0; i2 = 0; i3 = 0;         s0 = 0; s1 = 0;
        #5              i0 = 1; i1 = 0; i2 = 0; i3 = 0;         s0 = 0; s1 = 0;
        #5              i0 = 0; i1 = 1; i2 = 0; i3 = 0;         s0 = 1; s1 = 0;
        #5              i0 = 0; i1 = 0; i2 = 1; i3 = 0;         s0 = 0; s1 = 1;
        #5              i0 = 0; i1 = 0; i2 = 0; i3 = 1;         s0 = 1; s1 = 1;
```

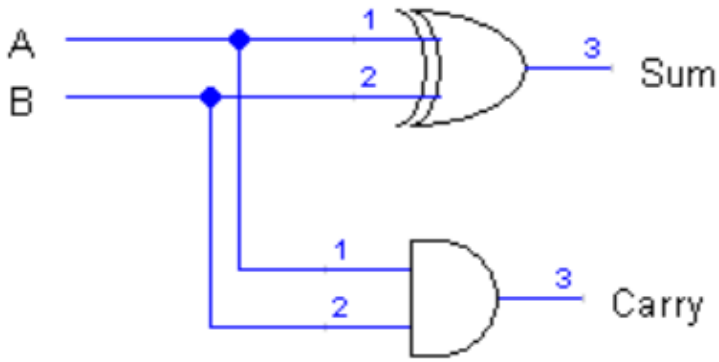
```
        #10     $finish;
        end
endmodule
```

Waveform



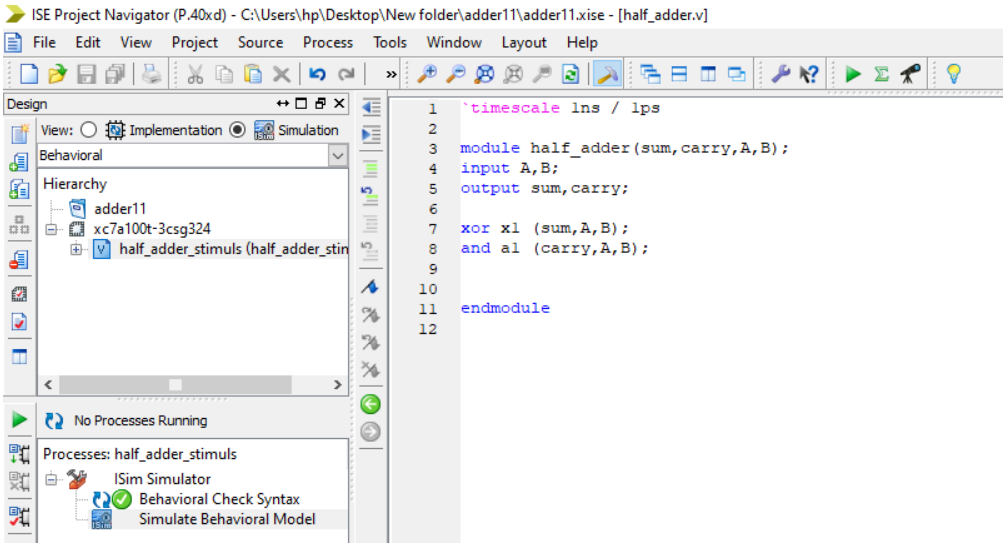
Example # 3

Implement half adder (Gate level designing) in Xilinx. implement test benches to verify their working. Visualize their behavior in waveforms.

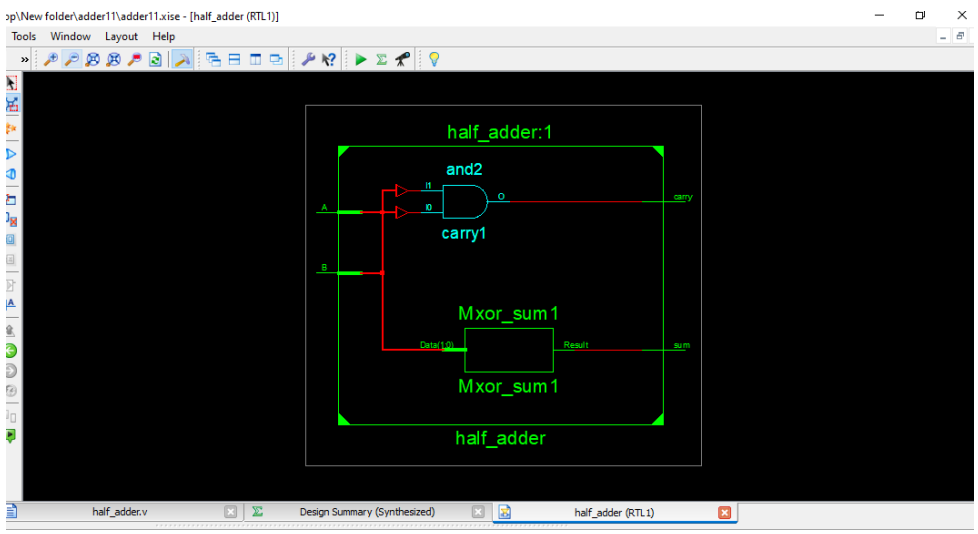


Code

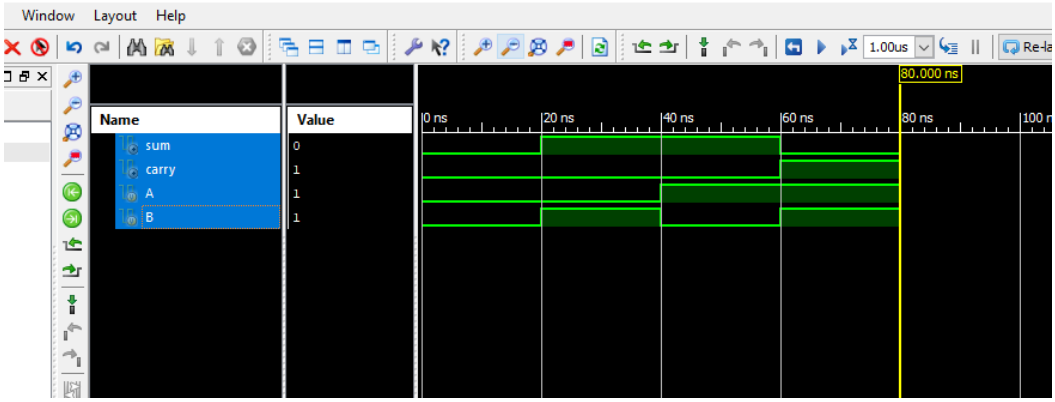
```
module half_adder(sum,carry,A,B);
input A,B;
output sum,carry;
xor x1 (sum,A,B);
and a1 (carry,A,B);
endmodule
```



RTL

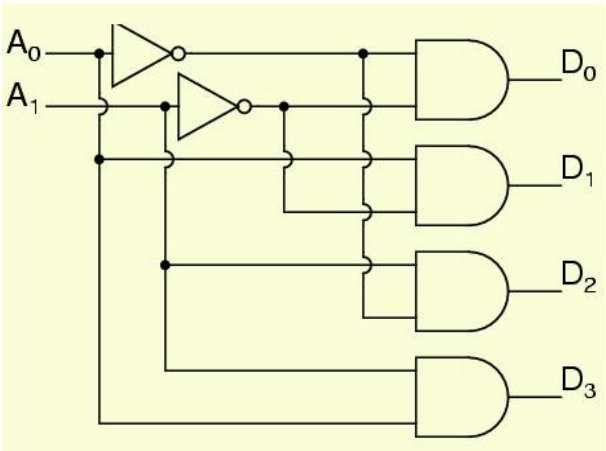


Waveform



Example # 4

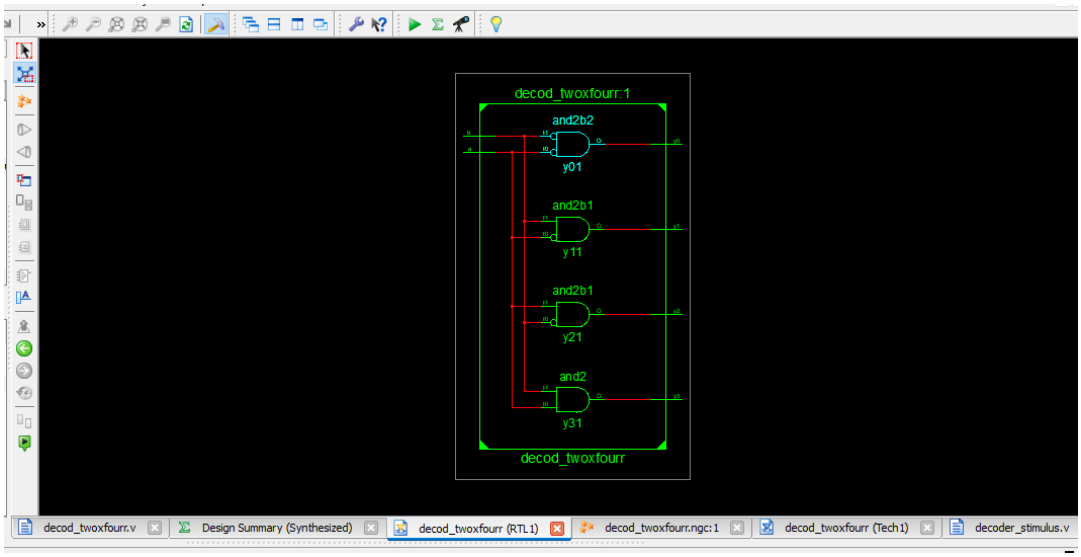
Implement **2to4** decoder (Gate level designing) in Xilinx. implement test benches to verify their working. Visualize their behavior in waveforms.



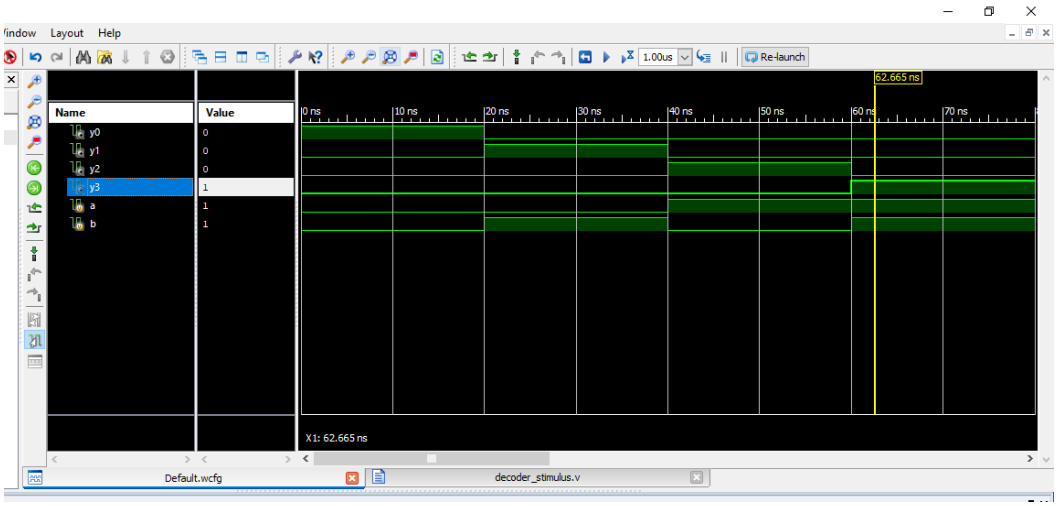
Code

```
module decod_twoxfourr(y0,y1,y2,y3,a,b);
input a,b;
output y0,y1,y2,y3;
wire abar,bbar;
not (abar,a);
not (bbar,b);
and (y0,abar,bbar);
and (y1,abar,b);
and (y2,a,bbar);
and (y3,a,b);
endmodule
```

RTL



Waveform



Lab Tasks:

Implement the following in Xilinx (Gate Level Designing). implement test benches to verify their working. Visualize their behavior in waveforms.

1. Full adder
2. Full subtractor
3. 8 X 1 multiplexer

Home tasks:

4. 4 to 2 encoders
5. 1 to 4 demultiplexer

Lab 3: Verilog Behavioral Modeling

Objectives:

- Implement a 4-bit ALU in Verilog for basic arithmetic and logical operations.
- Design a 32-bit ALU capable of advanced operations.
- Use always and initial statements for structured procedures in behavioral modeling.
- Apply conditional statements for decision-making in Verilog.
- Explore event control with the @ symbol to trigger statements based on signal changes.
- Understand the practical use of arrays in scenarios like multiplexers and decoders.

Introduction:

Verilog provides designers with the ability to describe design functionality in an algorithmic manner. In other words, the designer describes the **behavior** of the circuit. Thus, behavioral modeling represents the circuit at a very high level of abstraction. Design at this level resembles C programming more than it resembles digital circuit design. Behavioral Verilog constructs are like C language constructs in many ways. Verilog is rich in behavioral constructs that provide the designer with a great amount of flexibility.

Structured Procedures:

There are two structured procedure statements in Verilog: *always* and *initial*. These statements are the two most basic statements in behavioral modeling. All other behavioral statements can appear only inside these structured procedure statements.
Each always and initial statement represents a separate activity flow in Verilog. Each activity flow starts at simulation time 0. The statements always and initial cannot be nested. The fundamental difference between the two statements is explained in the following sections.

a) Initial Statement

All statements inside an initial statement constitute an **initial block**. An initial block starts at time 0, executes exactly once during a simulation, and then does not execute again. If there are multiple initial blocks, each block starts to execute concurrently at time 0. Each block finishes execution independently of other blocks. Multiple behavioral statements must be grouped; typically using the keywords begin and end. If there is only one behavioral statement, grouping is not necessary. This is similar to the { } grouping in the C programming language.
Example:

```
initial
m = 1'b0; //single statement; does not need to be grouped
initial begin
#5 a = 1'b1; //multiple statements; need to be grouped
#25 b = 1'b0;
end
```

```
initial begin
#10 x = 1'b0;
#25 y = 1'b1;
end
initial
#50 $finish;
```

In the above example, the three initial statements start to execute in parallel at time 0. If a delay #<delay> is seen before a statement, the statement is executed <delay> time units after the current simulation time. Thus, the execution sequence of the statements inside the initial blocks will be as follows.

```
time      statement executed 0 m = 1'b0;
5    a = 1'b1;
10   x = 1'b0;
30   b = 1'b0;
35   y = 1'b1;
50   $finish;
```

The initial blocks are typically used for initialization, monitoring, waveforms and other processes that must be executed only once during the entire simulation run.

b) Always Statement

All behavioral statements inside an always statement constitute an **always block**. The always statement starts at time 0 and executes the statements in the always block continuously in a looping fashion. This statement is used to model a block of activity that is repeated continuously in a digital circuit. An example is a clock generator module that toggles the clock signal every half cycle. In real circuits, the clock generator is active from time 0 to as long as the circuit is powered on.

Example:

```
//Initialize clock at time zero
initial
clock = 1'b0;

//Toggle clock every half-cycle
always
#10 clock = ~clock;
initial
#1000 $finish;
```

In the example, the always statement starts at time 0 and executes the statement clock = ~clock every 10-time units. Notice that the initialization of clock must be done inside a separate initial statement. If we put the initialization of clock inside the always block, clock will be initialized every time the always is entered. Also, the simulation must be halted inside an initial statement. If there is no $stop or $finish statement to halt the simulation, the clock generator will run forever.

C programmers might draw an analogy between the always block and an infinite loop. But hardware designers tend to view it as a continuously repeated activity in a digital circuit starting from power on. The activity is stopped only by power off ($finish) or by an interrupt ($stop).

Conditional Statements:

Conditional statements are used for making decisions based upon certain conditions. These conditions are used to decide whether or not a statement should be executed. Keywords if and else are used for conditional statements. There are three types of conditional statements. Usage of conditional statements is shown below.

```
//Type 1 conditional statement. No else statement
//Statement executes or does not execute
if (<expression>) true_statement ;

//Type 2 conditional statement. One else statement
//Either true_statement or false_statement is evaluated
if (<expression>) true_statement ; else false_statement ;

//Type 3 conditional statement. Nested if-else-if
//Choice of multiple statements. Only one is executed.
if (<expression1>) true_statement1 ;
else if (<expression2>) true_statement2 ;
else if (<expression3>) true_statement3 ;
else default_statement ;
```

The <*expression*> is evaluated. If it is true (1 or a non-zero value), the *true_statement* is executed. However, if it is false (zero) or ambiguous (x), the *false_statement* is executed. The <*expression*> can contain any operators used in dataflow modeling. Each *true_statement* or *false_statement* can be a single statement or a block of multiple statements. A block must be grouped, typically by using keywords **begin** and **end**. A single statement need not be grouped.

Multiway Branching

In type 3 conditional statement in the previous section, there were many alternatives, from which one was chosen. The nested if-else-if can become unwieldy if there are too many alternatives. A shortcut to achieve the same result is to use the case statement.
The keywords case, endcase, and default are used in the case statement.

```
case (expression)
alternative1: statement1;
alternative2:  statement2;
alternative3:  statement3;
...
...
default: default_statement;
```

```
endcase
```

Each of *statement1, statement2 …, default_statement* can be a single statement or a block of multiple statements. A block of multiple statements must be grouped by keywords **begin** and **end**. The expression is compared to the alternatives in the order they are written. For the first alternative that matches, the corresponding statement or block is executed. If none of the alternatives matches, the *default_statement* is executed. The *default_statement* is optional. Placing of multiple default statements in one case statement is not allowed. The case statements can be nested. The case statement can also act like a many-to-one multiplexer

Event control

The @ symbol is used to specify an event control. Statements can be executed on changes in signal value or at a positive or negative transition of the signal value. The keyword posedge is used for a positive transition, as shown in example.

Example

```
@(clock) q = d; //q = d is executed whenever signal clock changes value
@(posedge clock) q = d; //q = d is executed whenever signal clock does a
positive transition
```

Event OR Control

Sometimes a transition on any one of multiple signals or events can trigger the execution of a statement or a block of statements. This is expressed as an OR of events or signals. The list of events or signals expressed as an OR is also known as a sensitivity list. The keyword or is used to specify multiple triggers, as shown in Example.

Example

```
always @( reset or clock or d) //Wait for reset or clock or d to change
begin
if (reset) //if reset signal is high, set q to 0
q = 1'b0;
else if(clock) //if clock is high, q = d
q = d;
end
```

Sensitivity lists can also be specified using the "," (comma) operator instead of the or operator. Comma operators can also be applied to sensitivity lists that have edge-sensitive triggers. The above example can be rewritten using the comma operator, the only change would be:

always @(reset, clock, d)

When the number of input variables to a combination logic block are very large, sensitivity lists can become very cumbersome to write. To solve this problem, Verilog HDL contains two special symbols: @* and @(*). Both symbols exhibit identical behavior. These special symbols are sensitive to a change on any signal that may be read by the statement group that follows this symbol.

```
//Combination logic block using the or operator
```

```
//Cumbersome to write and it is easy to miss one input to the block
always @(a or b or c or d or e)
begin
out1 = a ? b+c : d+e;
end
```

Instead of the above method, use @(*) symbol. Alternately, the @* symbol can be used
All input variables are automatically included in the sensitivity list

```
always @(*)
begin
out1 = a ? b+c : d+e;
end
```

The **concatenation operator** "{ , }" combines (concatenates) the bits of two or more data objects. The objects may be scalar (single bit) or vectored (muliple bit).

Suppose we have two 2-bit binary numbers:

Binary number A: 10

Binary number B: 01

Now, let's concatenate these two numbers:
Concatenation Result (4 bits): 1001
In this example, the concatenation of 10 (from A) and 01 (from B) results in the 4-bit binary number 1001. The leftmost two bits come from A, and the rightmost two bits come from B.

Examples:

Example 1: And gate (Behavioral modelling)
Code

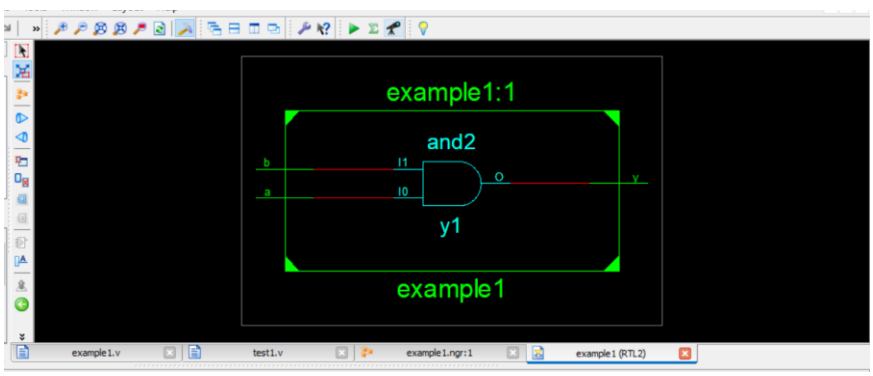
```
module example1 (a,b,y
  );
      input a,b;
      output reg y;

always @ (a , b)
begin
      if (a==1 && b==1) y = 1;
      else y = 0;
end
endmodule
```

Test Bench

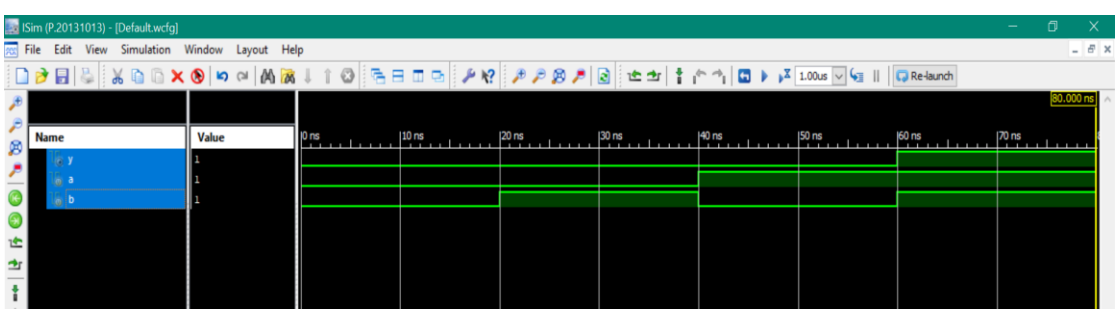
```
module test1;
```

```
reg a;
reg b;

wire y;

example1 uut (
        .a(a),
        .b(b),
        .y(y)
);

initial
        begin
        a = 0; b = 0;
        #20  a= 0; b = 1;
        #20  a= 1; b = 0;
        #20  a= 1; b = 1;
        #20 $finish;
        end

endmodule
```

RTL



Waveform



Example # 2 Half Adder (Behavioral modelling)

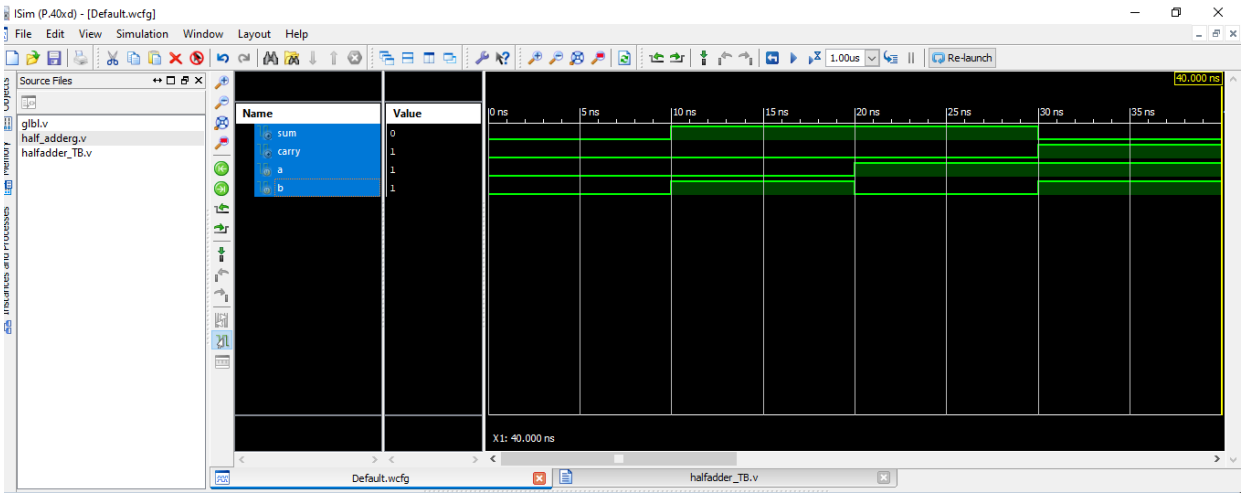
Code

```
module half_adder(sum,carry,a,b);
input a,b;
output sum,carry;
reg sum,carry;
// always(*)
always@(a or b)
begin
sum = a^b;
carry = a&b;
end
endmodule
```

Test Bench

```
module halfadder_TB;
    // Inputs
    reg a;
    reg b;
    // Outputs
    wire sum;
    wire carry;
    half_adder gate(sum,carry,a,b);

    initial
    begin
    $monitor($time, "sum=%b,carry=%b,a=%b,b=%b",sum,carry,a,b);
    a = 0; b = 0;
    #10     a = 0; b = 1;
    #10     a = 1; b = 0;
    #10     a = 1; b = 1;
    #10 $finish;
    end
    endmodule
```

Waveform



Console window result

```
Console
INFO: There is another simulation running in the same directory. Using database file name isim1.wdb.
ISim P.40xd (signature 0x7708f090)
This is a Full version of ISim.
Time resolution is 1 ps
Simulator is doing circuit initialization process.
Finished circuit initialization process.
          0sum=0,carry=0,a=0,b=0
          10sum=1,carry=0,a=0,b=1
          20sum=1,carry=0,a=1,b=0
          30sum=0,carry=1,a=1,b=1
Stopped at time : 40 ns : File "C:/Users/hp/Desktop/New folder/l3adder/halfadder_TB.v" Line 49
ISim> |
```

Example 3: 4-to-1 Multiplexer with Case Statement

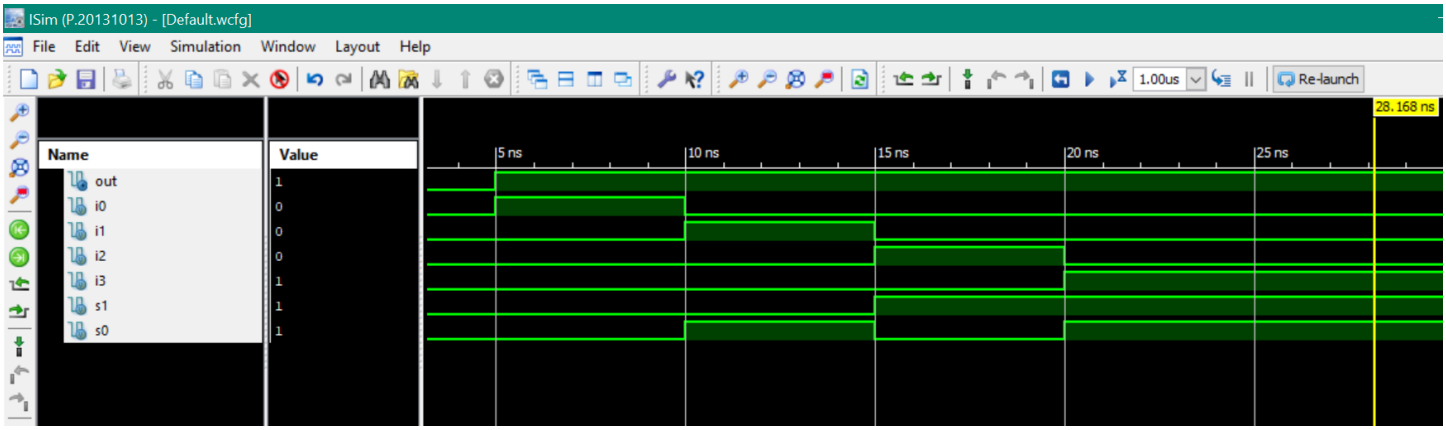
```
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);
// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3; input s1, s0;
reg out;
always @ (s1 or s0 or i0 or i1 or i2 or i3)
case ({s1, s0}) //Switch based on concatenation of control signals
2'd0 : out = i0;
2'd1 : out = i1;
2'd2 : out = i2;
2'd3 : out = i3;
default: $display("Invalid control signals");
endcase
```

```
endmodule
```

Test Bench:

```
module mux4_to_1_test;

	// Inputs
	reg i0;
	reg i1;
	reg i2;
	reg i3;
	reg s1;
	reg s0;

	// Outputs
	wire out;

	// Instantiate the Unit Under Test (UUT)
	mux4_to_1 uut (
		.out (out),
		.i0 (i0),
		.i1 (i1),
		.i2 (i2),
		.i3 (i3),
		.s1 (s1),
		.s0 (s0)
	);

	initial begin

			i0 = 0; i1 = 0; i2 = 0; i3 = 0;	s0 = 0; s1 = 0;
	#5		i0 = 1; i1 = 0; i2 = 0; i3 = 0;	s0 = 0; s1 = 0;
	#5		i0 = 0; i1 = 1; i2 = 0; i3 = 0;	s0 = 1; s1 = 0;
	#5		i0 = 0; i1 = 0; i2 = 1; i3 = 0;	s0 = 0; s1 = 1;
	#5		i0 = 0; i1 = 0; i2 = 0; i3 = 1;	s0 = 1; s1 = 1;
	#10	$finish;

	end

endmodule
```

Waveform:



Example # 4 {2_to_4} Decoder (Behavioral modelling)

Code

```
module decoder_2to4(in,y);
input [1:0]in;
output reg[3:0]y;

always@(in)
begin
if(in==2'b00)
  y = 4'b0001;
else if(in==2'b01)
  y = 4'b0010;
else if(in==2'b10)
  y = 4'b0100;
else if(in==2'b11)
  y = 4'b1000;
end
endmodule
```

Test Bench

```
module decoder_2to4_stimulus;

// Inputs
reg [1:0] in;

// Outputs
wire [3:0] y;

// Instantiate the Unit Under Test (UUT)
decoder_2to4 uut (
        .in(in),
```

```
        .y(y)
);

initial
begin
$monitor($time , "in=%b,y=%b",in,y);
        // Initialize Inputs
      in = 2'b00;
#4    in = 2'b01;
#4    in = 2'b10;
#4    in = 2'b11;
#2    $finish;
end
endmodule
```

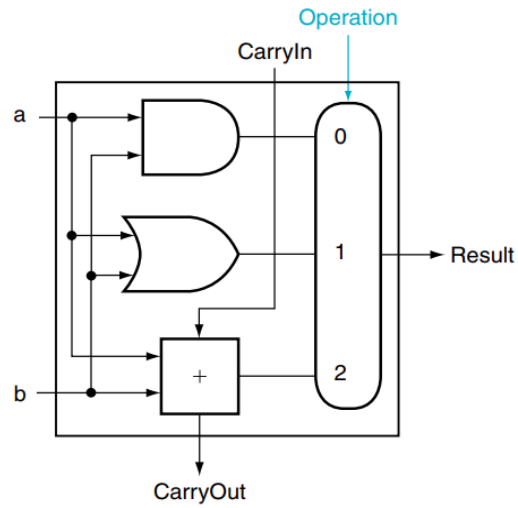
Waveform

Arithmetic Logic Unit:

An ALU (Arithmetic Logic Unit) is a fundamental component of a CPU (Central Processing Unit) responsible for performing arithmetic and logic operations on binary data. It takes two input operands and produces a result based on the specified operation.

Example # 5: Simple 1-bit ALU that performs the following operations on specific signals:

Control Signal	Operation
00	AND
01	OR
10	Addition



Code:

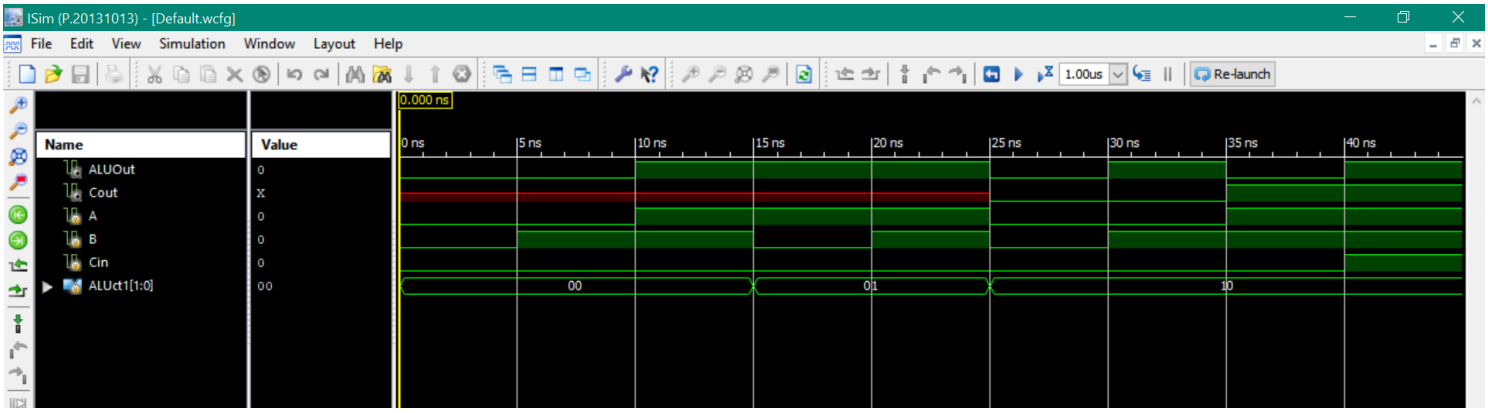
```
module example5 (
   input A,B, Cin,
         input [1:0] ALUct1,
   output reg ALUOut, Cout
   );

always @(ALUct1, A, B, Cin)
         begin
                  case (ALUct1)
                  0: ALUOut <= A&B;
                  1: ALUOut <= A|B;
                  2: {Cout, ALUOut} <= Cin+A+B;
                  default: ALUOut <= 0;
                  endcase
         end
endmodule
```

Test Bench:

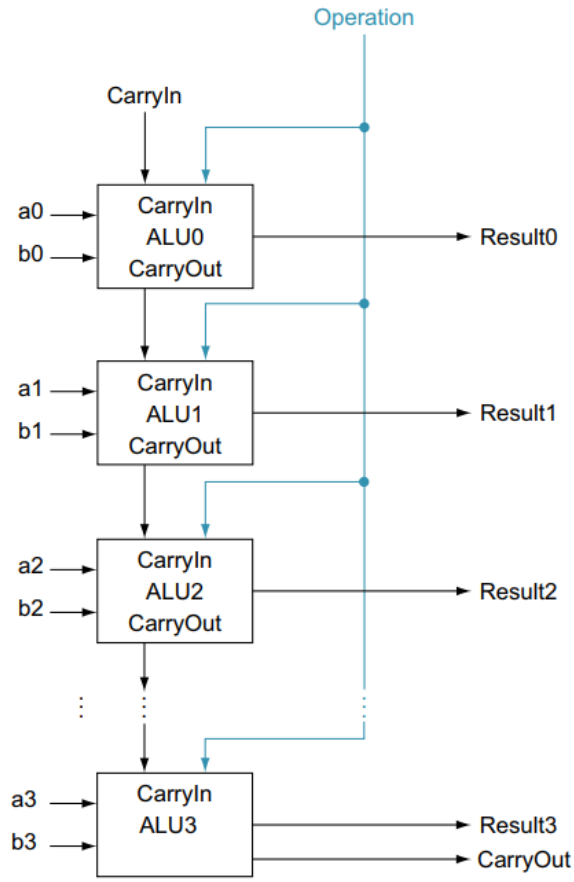
```
module test_example5;

         // Inputs
         reg A;
         reg B;
         reg Cin;
         reg [1:0] ALUct1;

         // Outputs
         wire ALUOut;
         wire Cout;
```

```
        // Instantiate the Unit Under Test (UUT)
        example5 uut (
                .A(A),
                .B(B),
                .Cin(Cin),
                .ALUct1(ALUct1),
                .ALUOut(ALUOut),
                .Cout(Cout)
        );

        initial begin
                // AND
                A = 0; B = 0; Cin = 0;        ALUct1 = 0;
#5              A = 0; B = 1;
#5              A = 1; B = 1;

                // OR
#5              A = 1; B = 0; ALUct1 = 1;
#5              A = 1; B = 1;

                // Full Adder
#5              A = 0; B = 0; ALUct1 = 2;
#5              A = 0; B = 1;
#5              A = 1; B = 1;
#5              A = 1; B = 1; Cin = 1;
#5              $finish;

        end

endmodule
```

Waveform:



Example 6: 4-bit ALU using instantiation from example 5.



Code:

```
module example6 (
   input [3:0] A,B,
         input Cin,
         input [1:0] ALUct1,
   output [3:0] ALUOut,
         output Cout
   );

         wire c1, c2, c3;

example5_alu alu1 (A[0], B[0], Cin, ALUct1, ALUOut[0], c1);
example5_alu alu2 (A[1], B[1], c1, ALUct1, ALUOut[1], c2);
example5_alu alu3 (A[2], B[2], c2, ALUct1, ALUOut[2], c3);
example5_alu alu4 (A[3], B[3], c3, ALUct1, ALUOut[3], Cout);

endmodule
```

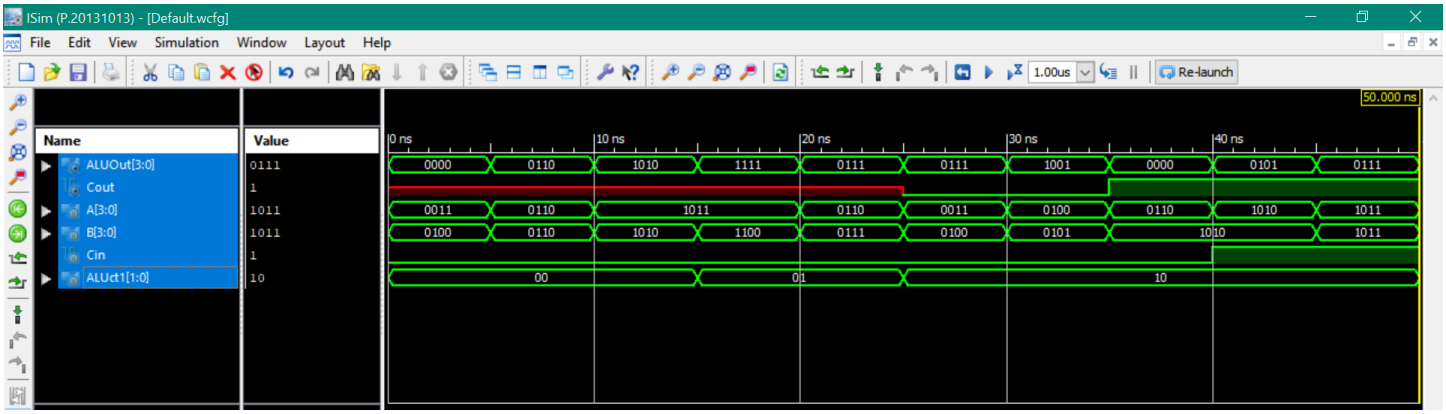
```
module example5_alu (
   input A, B, Cin,
         input [1:0] ALUct1,
   output reg ALUOut, Cout
   );

always @(ALUct1, A, B, Cin)
        begin
                case (ALUct1)
                0: ALUOut <= A&B;
                1: ALUOut <= A|B;
                2: {Cout, ALUOut} <= Cin+A+B;
                default: ALUOut <= 0;
                endcase
        end
endmodule
```

TestBench:

```
module ex6_test;

        // Inputs
        reg [3:0] A;
        reg [3:0] B;
        reg Cin;
        reg [1:0] ALUct1;

        // Outputs
        wire [3:0] ALUOut;
        wire Cout;

        // Instantiate the Unit Under Test (UUT)
        example6 uut (
                .A (A),
                .B (B),
                .Cin (Cin),
                .ALUct1 (ALUct1),
                .ALUOut (ALUOut),
                .Cout (Cout)
        );

        initial begin
                // Initialize Inputs

                // AND
                A = 3; B = 4; Cin = 0;        ALUct1 = 0;
```

```
#5          A = 6; B = 6;
#5          A = 11; B = 10;

            // OR
#5          A = 11; B = 12; ALUct1 = 1;
#5          A = 6; B = 7;

            // Full Adder
#5          A = 3; B = 4; ALUct1 = 2;
#5          A = 4; B = 5;
#5          A = 6; B = 10;
#5          A = 10; B = 10; Cin = 1;
#5          A = 11; B = 11; Cin = 1;
#5          $finish;

        end

endmodule
```

Waveform:



Lab Tasks:

1. Design an 8x1 multiplexer using behavioral modeling.

2. Extend the 4-bit ALU in example 6 with more operations that are given below.

Control Signal	Operation
0	AND
1	OR
2	Addition
6	Subtraction
7	Check if A < B *
12	NOR

*Output will be 1 if A is less than B, else output will be 0.

3. Design a 32-bit ALU in Verilog that can perform similar operations as in task 2 using behavioral modeling. Add an extra output 'Zero' that will be 1 if ALUOut is 0.

Lab 4: Sequential Circuits in Verilog HDL

Objectives:

- Differentiate between combinational and sequential circuits effectively.
- Gain insight into the diverse types of flip-flops.
- Implement various flip-flops using Verilog.
- Create a 32-bit register in Verilog to demonstrate practical register design.
- Develop and implement a 32-bit up counter in Verilog for sequential counting operations.
- Combine program counter and adder together using instantiation.

Introduction:

Digital logic circuits can be classified into "combinational" and "sequential". A combinational logic circuit is one whose output solely depends on its current inputs. sequential circuits, on the other hand, are built using combinational circuits and memory elements called "flip-flops". These circuits generate output that depends on the current and previous states. A Digital logic (or switching) circuit has voltage levels to be switched from one value to another but has a finite number of distinct values (generally 0 for false and 1 for true). These circuits operate a definite set of logic rules, so it is also known as Logic circuits. These are basic circuits that are used in mobile phones, calculators, computers, etc.

Types of Logic Circuits: There are two types of Digital circuits depending on their output and memory used:

(i) Combinational circuit, and

(ii) Sequential circuit

A combinational circuit consists of logic gates whose outputs at any time are determined from only the present combination of inputs and they have no memory. A sequential circuit consists of logic gates whose outputs at any time are determined from both the present combination of inputs and previous output. That means sequential circuits use memory elements to store the value of previous output.

Combinational Circuits: These circuits are developed using AND, OR, NOT, NAND, and NOR logic gates. These logic gates are building blocks of combinational circuits. A combinational circuit consists of input variables and output variables. Since these circuits are not dependent upon previous input to generate any output, so are combinational logic circuits. A combinational circuit can have an n number of inputs and m number of outputs. In combinational circuits, the output at any time is a direct function of the applied external inputs.

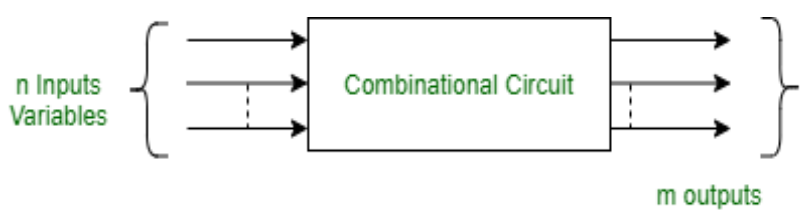


Figure - Block diagram of Combinational circuit

Sequential circuits: A sequential circuit is specified by a time sequence of inputs, outputs, and internal states. The output of a sequential circuit depends not only on the combination of present inputs but also on the previous outputs. Unlike combinational circuits, sequential circuits include memory elements with combinational circuits. Examples are Flipflops i.e., S R Flipflop, J K Flipflop, T-Flipflop and D-Flipflops.

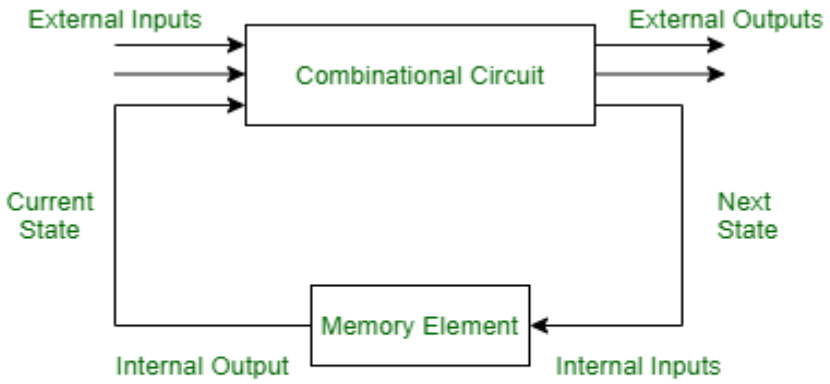


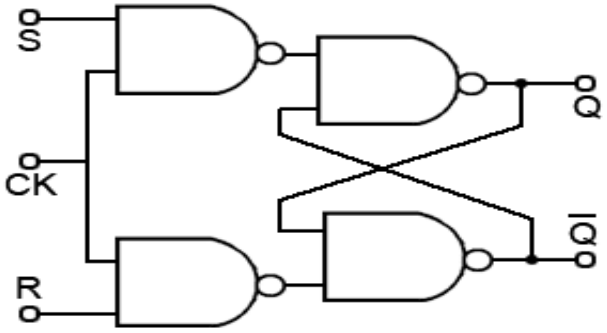
Figure - Sequential Circuit

- The memory elements are circuits capable of storing binary information.
- The binary information stored in these memory elements at any given time defines the state of the sequential circuit at that time.
- The external output of a sequential circuit depends both on the present input and the previous output state.
- The next state of the memory elements also depends on the external input and the present state of the external output.
- Some sequential circuits may not contain combinational circuits, but only memory elements.

Examples:

Example # 1 S R Flip Flop

Block Diagram



Truth Table (S R Flip-flop)

Inputs		Outputs	
S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	invalid	
1	1	invalid	

Code

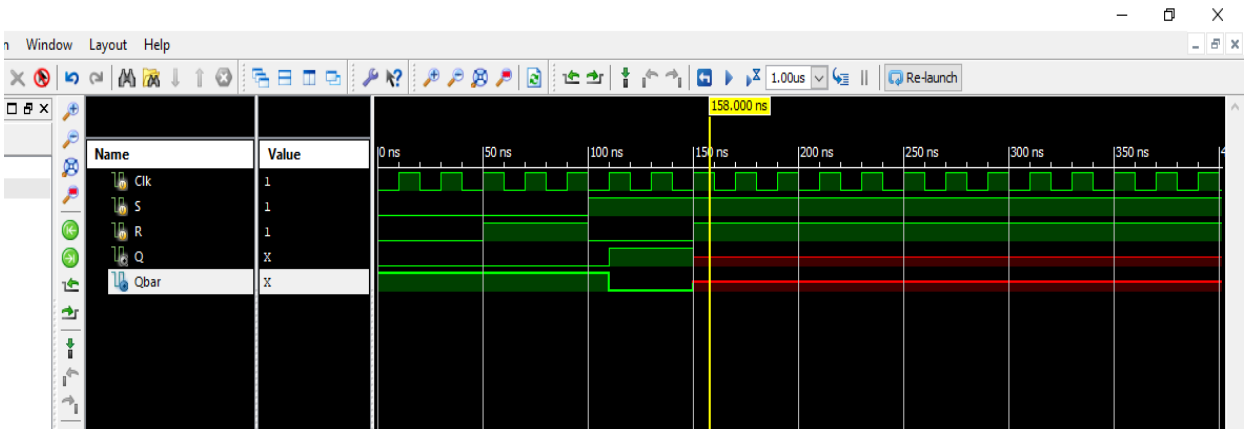
```
module sr_flipflop(Q,Qbar,S,R,Clk);
input S,R,Clk;
output reg Q, Qbar;

always @ (posedge Clk)
begin
case ({S,R})
2'b00:Q = Q;
2'b01:Q = 1'b0;
2'b10:Q = 1'b1;
2'b11:Q = 1'bx;
endcase
Qbar = ~Q;
end
endmodule
```

Test Bench

```
module srff_TB;
// Inputs
reg S;
reg R;
reg Clk;
// Outputs
wire Q;
```

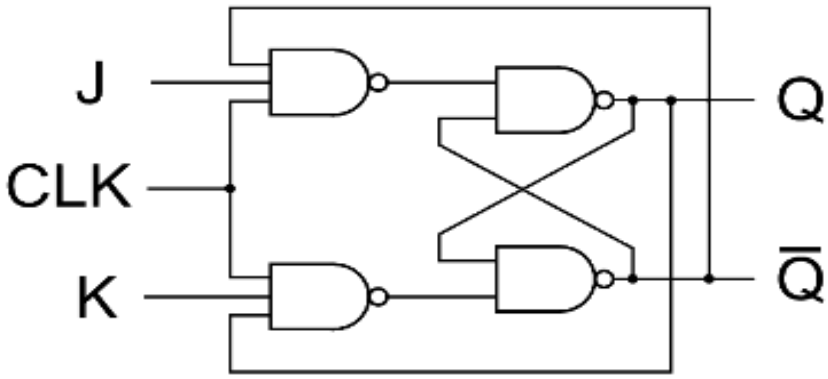
```
wire Qbar;
// Instantiate the Unit Under Test (UUT)
sr_flipflop uut (.Q(Q), .Qbar(Qbar), .S(S), .R(R), .Clk(Clk));
initial begin
    S = 0; R = 0; Clk = 0;
#50         S = 0; R = 1;
#50         S = 1; R = 0;
#50         S = 1; R = 1;
end
 always # 10 Clk = ~Clk;
endmodule
```

Wave form



Example # 2 J K Flip flop

Circuit Diagram



Truth Table

J	K	Q_n	Q_{n+1}	State
0	0	0	0	Q_n (Hold)
0	0	1	1	
0	1	0	0	Reset
0	1	1	0	
1	0	0	1	Set
1	0	1	1	
1	1	0	1	Toggel
1	1	1	0	

Code

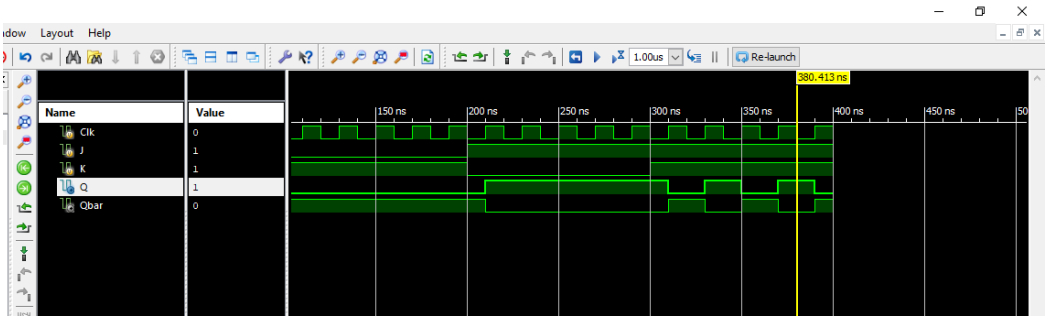
```
module JK_FF(Q,Qbar,Clk,J,K);
input J,K, Clk;
output Q,Qbar;
reg Q, Qbar ;

always @ (posedge Clk)
begin

    case({J,K})
    2'b00:Q = Q;
    2'b01:Q = 1'b0;
    2'b10:Q = 1'b1;
    2'b11:Q = ~Q;
    endcase
    Qbar = ~Q;
end
endmodule
```

Test Bench

```
module JK_FF_TB;

// Inputs
reg Clk;
reg J;
reg K;
```

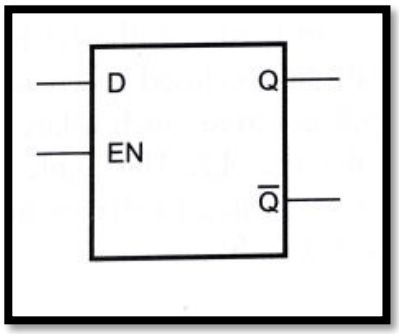
```
// Outputs
wire Q;
wire Qbar;

// Instantiate the Unit Under Test (UUT)
JK_FF uut (.Q(Q), .Qbar(Qbar), .Clk(Clk), .J(J), .K(K));
initial
begin
// Initialize Inputs
Clk = 0; J = 0; K = 0;
#100  J = 0 ; K = 1;
#100  J = 1 ; K = 0;
#100  J = 1 ; K = 1;
#100  $finish;
end
always #10 Clk = ~Clk;
endmodule
```

Waveform

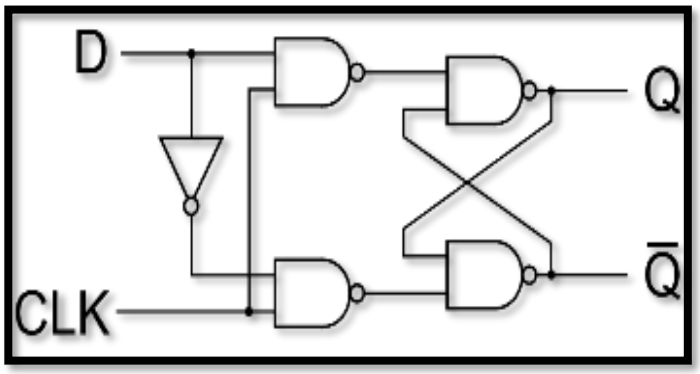


D Flip-Flop:

D flip-flop is a better alternative that is very popular with digital electronics. They are commonly used for counters and shift-registers and input synchronization. The **logic symbol** of D flip-flop is shown in the following figure.



D flip-flop operates with only positive clock transitions or negative clock transitions. Whereas D latch operates with enable signal. That means, the output of D flip-flop is insensitive to the changes in the input, D except for active transition of the clock signal. The **circuit diagram** of D flip-flop is shown below:



D Flip-Flop using NAND Gates

This circuit has single input D and two outputs Q(t) & Q(t)'. The operation of D flip-flop is similar to D Latch. But this flip-flop affects the outputs only when positive transition of the clock signal is applied instead of active enable.

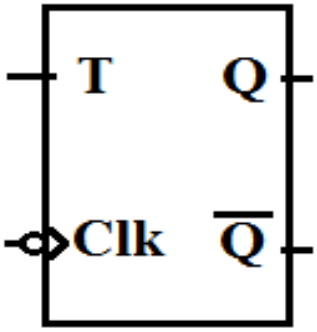
Truth Table

The following table shows the **state table** of D flip-flop.

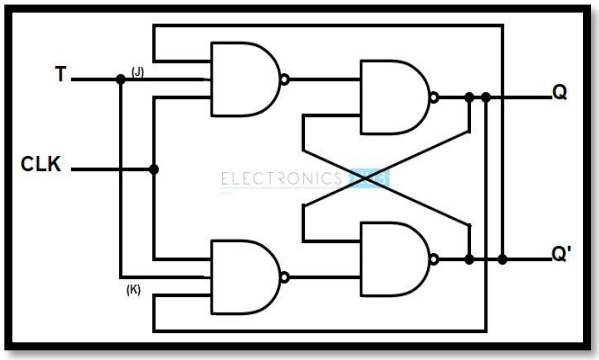
D	Q(t + 1)
0	0
1	1

T Flip-Flop:

T flip-flop is the simplified version of JK flip-flop. It is obtained by connecting the same input 'T' to both inputs of JK flip-flop. It operates with only positive clock transitions or negative clock transitions. The **logic symbol** of T flip-flop is shown in the following figure.



A T flip-flop is like a JK flip-flop. These are basically a single input version of JK flip-flops. This modified form of JK flip-flop is obtained by connecting both inputs J and K together. It has only one input along with the clock input.



JK Flip-Flop using NAND Gates

This circuit has single input T and two outputs Q(t) & Qt()'. The operation of T flip-flop is same as that of JK flip-flop. Here, we considered the inputs of JK flip-flop as **J = T** and **K = T** in order to utilize the modified JK flip-flop for 2 combinations of inputs. So, we eliminated the other two combinations of J & K, for which those two values are complement to each other in T flip-flop.

- **State Table:**

The following table shows the **state table** of T flip-flop.

D	**Q**(t+1)
0	Q(t)
1	Q(t)'

Here, Q(t) & Q(t+1) are present state & next state respectively. So, T flip-flop can be used for one of these two functions such as Hold, & Complement of present state based on the input conditions, when positive transition of clock signal is applied. The following table shows the **characteristic table** of T flip-flop.

Inputs	**Present State**	**Next State**
T	**Q(t)**	**Q(t+1)**
0	0	0
0	1	1
1	0	1

1	1	0

The output of T flip-flop always toggles for every positive transition of the clock signal when input T remains at logic High 1. Hence, T flip-flop can be used in **counters**.

Example 3: D Flip-Flop:

Code:

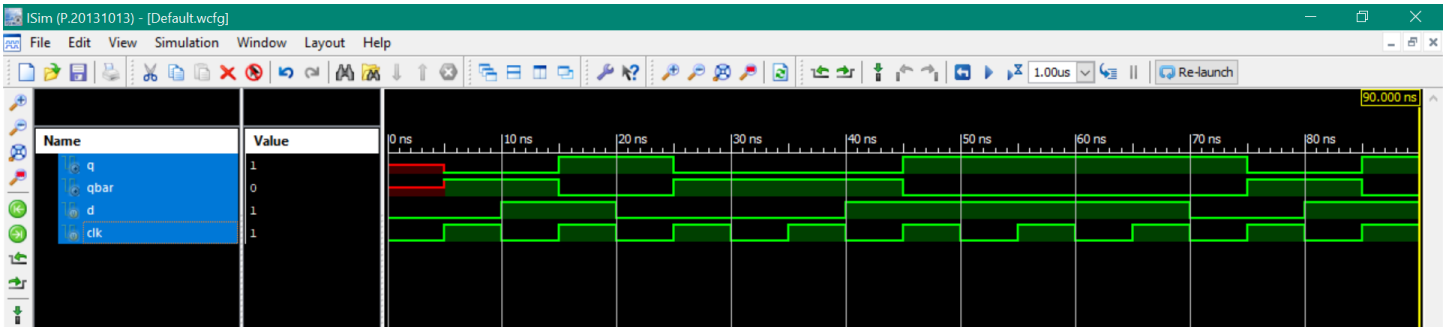
```
module nand_gate (c, a, b) ;
input a, b;
output c;
assign c = ~ (a&b) ;
endmodule

module not_gate (f, e) ;
input e;
output f;
assign f= ~e;
endmodule

module dff_using_struct (q, qbar, d, clk) ;
input d, clk;
output q, qbar;
not_gate not1 (dbar, d) ;
nand_gate nand1 (x, clk, d) ;
nand_gate nand2 (y, clk, dbar) ;
nand_gate nand3 (q, qbar, x) ;
nand_gate nand4 (qbar, q, y) ;
endmodule
```

TestBench:

```
module test_dff;

    // Inputs
    reg d;
    reg clk;

    // Outputs
    wire q;
    wire qbar;

    // Instantiate the Unit Under Test (UUT)
```

```
        dff_using_struct uut (
                .q(q),
                .qbar(qbar),
                .d(d),
                .clk(clk)
        );

always #5 clk = ~clk;
        initial begin
                // Initialize Inputs
                d = 0;
                clk = 0;

                #10 d = 1;
                #10 d = 0;
                #10 d = 0;
                #10 d = 1;
                #10 d = 1;
                #10 d = 1;
                #10 d = 0;
                #10 d = 1;
                #10 $finish;

        end

endmodule
```

Waveform:



Example: 4-bit Down Counter:

Code:

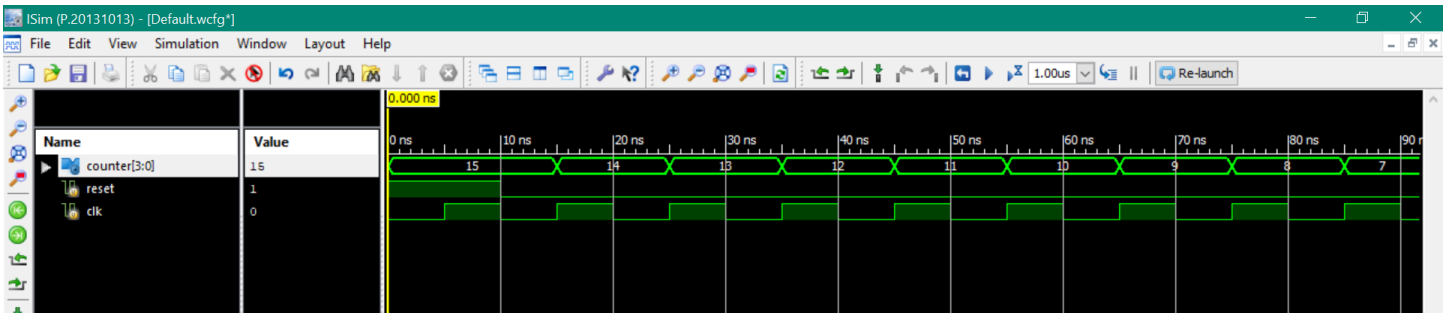
```
module counter (
input reset, clk,
output reg [3:0] counter
   );
```

```
always @ (posedge clk, posedge reset)
begin
if (reset)
        counter = 4'b1111;
else
        counter = counter - 1;
end

endmodule
```

Test Bench:

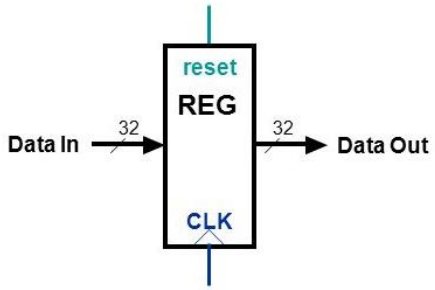
```
module test;

        reg reset;
        reg clk;
        wire [3:0] counter;

        counter uut (
                .reset (reset),
                .clk (clk),
                .counter (counter)
        );

        always #5 clk = ~clk;

        initial begin
                reset = 1;
                clk = 0;
                #10 reset = 0;
                #300 $finish;
        end

endmodule
```

Waveform:



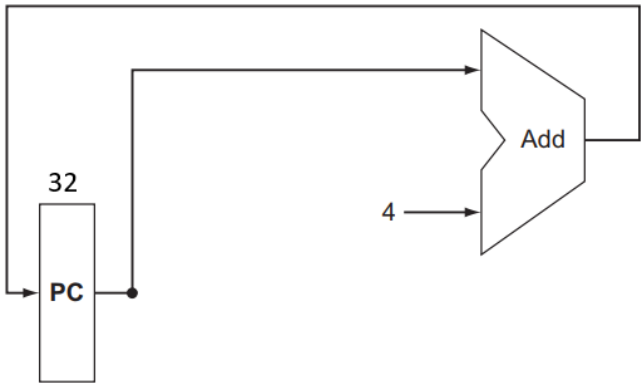
Lab Tasks:

1) *Create a 32-bit register in Verilog. On each positive edge of the clock, the 'Data Out' will be updated. Create a 'reset' input, that reset 'Data in', on every reset in sensitivity list.*



2) *Create a 32-bit up counter in Verilog. On each positive edge of the clock, the value will be updated by 1. Create a 'reset' input, that reset counter on every reset in sensitivity list.*

3) *Create a 32-bit counter that adds a 4-bit number into it on each positive edge of clock. A reset input will be used to reset the counter value to zero. Create counter and adder separately and connect them together using instantiation.*



Lab 5: Memories in Verilog

Lab Objectives:

- Implement a Verilog module for instruction memory.
- Understand the concept of instruction fetching in a processor using a program counter (PC).
- Initialize instruction memory with predefined instruction codes during reset.
- Verify instruction memory functionality through simulation, ensuring correct instruction fetching based on the program counter.
- Design a Verilog module for data memory with read and write functionality.
- Define inputs for write data, address, write enable and read enable.
- Implement logic to write data to memory.
- Develop logic for reading data from memory.
- Validate data memory operation through simulation and test bench, ensuring proper data storage and retrieval under different read and write scenarios.

Arrays declaration in Verilog:

In Verilog, arrays can be declared using the following syntax:

```
reg [n:0] array_name [m:0];
```

where:

- reg specifies the data type of the array. In this case, it is a register.
- [n:0] specifies the range of the array indices. In this example, n is the maximum index and 0 is the minimum index. {MSB and LSB respectively}
- array_name is the name of the array.
- [m:0] specifies the range of the array elements. In this example, m is the maximum array element value and 0 is the minimum.

For example, to declare a 4x4 2D array of 8-bit registers, you can use the following code:

```
reg [7:0] my_array [3:0][3:0];
```

This creates an array called my_array with 4 rows and 4 columns, where each element of the array is an 8-bit register. You can access an individual element of the array using its row and column index. For example, to assign a value of 7 to the element in the second row and third column, you can use the following code:

```
my_array[1][2] = 7;
```

Note that in Verilog, arrays are zero-indexed, meaning that the first element of an array is at index 0. Therefore, the indices for a 4x4 array range from 0 to 3.

Here's an example of a 2D array in Verilog:

```
module two_d_array();
```

```
 reg [7:0] array_2d [3:0][3:0];

initial
begin

  // Assigning values to the array elements

  array_2d[0][0] = 8'h11;
  array_2d[0][1] = 8'h22;
  array_2d[0][2] = 8'h33;
  array_2d[0][3] = 8'h44;
  array_2d[1][0] = 8'h55;
  array_2d[1][1] = 8'h66;
  array_2d[1][2] = 8'h77;
  array_2d[1][3] = 8'h88;
  array_2d[2][0] = 8'h99;
  array_2d[2][1] = 8'haa;
  array_2d[2][2] = 8'hbb;
  array_2d[2][3] = 8'hcc;
  array_2d[3][0] = 8'hdd;
  array_2d[3][1] = 8'hee;
  array_2d[3][2] = 8'hff;
  array_2d[3][3] = 8'h00;
  // Displaying values of the array elements
  $display("Array Elements:");
  for (int i = 0; i < 4; i = i + 1) begin
    for (int j = 0; j < 4; j = j + 1) begin
      $display("array_2d[%0d][%0d] = %h", i, j, array_2d[i][j]);
    end
  end
 end
endmodule
```

This code declares a 2D array called **array_2d** of size 4x4, where each element is an 8-bit register. The initial block assigns values to each element of the array, and for loops display the values of all the elements of the array.

The output of the code will be:

```
Array Elements:
array_2d[0][0] = 11
array_2d[0][1] = 22
array_2d[0][2] = 33
array_2d[0][3] = 44
array_2d[1][0] = 55
array_2d[1][1] = 66
```

array_2d[1][2] = 77
array_2d[1][3] = 88
array_2d[2][0] = 99
array_2d[2][1] = aa
array_2d[2][2] = bb
array_2d[2][3] = cc
array_2d[3][0] = dd
array_2d[3][1] = ee
array_2d[3][2] = ff
array_2d[3][3] = 00

This shows that the 2D array has been properly declared and initialized, and its elements can be accessed and manipulated as needed.

Concatenation in Verilog

In Verilog, concatenation is used to combine one or more values or variables into a single vector. The concatenation operator is represented by the **{}** symbol, and the values or variables to be concatenated are separated by commas inside the braces.

Here are a few examples of concatenation in Verilog:

Example 1: Concatenating two 4-bit registers to create an 8-bit register

```
reg [3:0] a, b;
reg [7:0] c;

// Assigning values to registers a and b

a = 4'b1110;
b = 4'b1111;

// Concatenating a and b to create c

c = {a, b}; // c = 8'b11101111
```

In this example, we have two 4-bit registers **a** and **b**. We use the concatenation operator **{}** to concatenate **a** and **b** and create an 8-bit register **c**. The resulting value of **c** is **8'b11101111**.

Example 2: Concatenating a 4-bit value and a 2-bit value to create a 6-bit value

```
reg [3:0] a;
reg [1:0] b;
reg [5:0] c;
// Assigning values to registers a and b
a = 4'b0110;
b = 2'b01;
// Concatenating a and b to create c
```

```
c = {a, b}; // c = 6'b011001
```

In this example, we have a 4-bit register a and a 2-bit register b. We use the concatenation operator {} to concatenate a and b and create a 6-bit register c. The resulting value of c is 6'b011001.

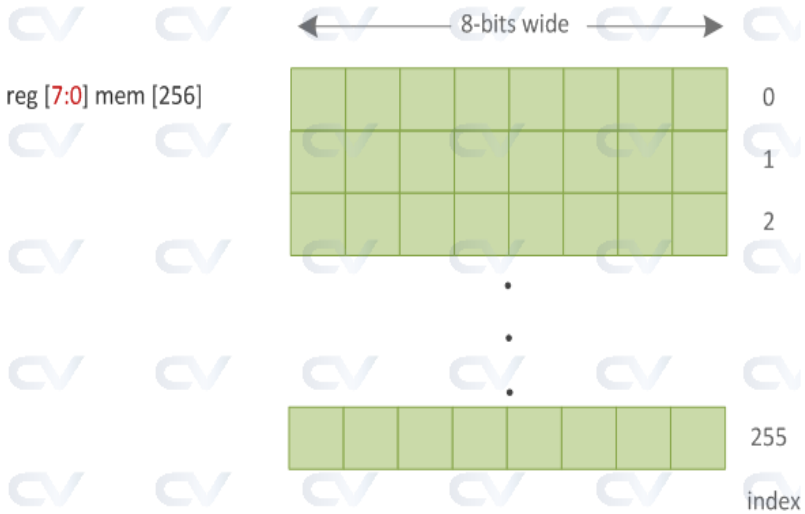
Example 3: Concatenating two vectors of different sizes

```
reg [7:0] a;
reg [3:0] b;
reg [11:0] c;
// Assigning values to registers a and b

a = 8'b10101010;
b = 4'b0110;

// Concatenating a and b to create c

c = {a, b}; // c = 12'b101010100110
```

In this example, we have an 8-bit register **a** and a 4-bit register **b**. We use the concatenation operator **{}** to concatenate **a** and **b** and create a 12-bit register **c**.
The resulting value of **c** is **12'b101010100110**.
Concatenation is a powerful feature in Verilog that allows you to create complex data structures and manipulate them efficiently.

Memories:

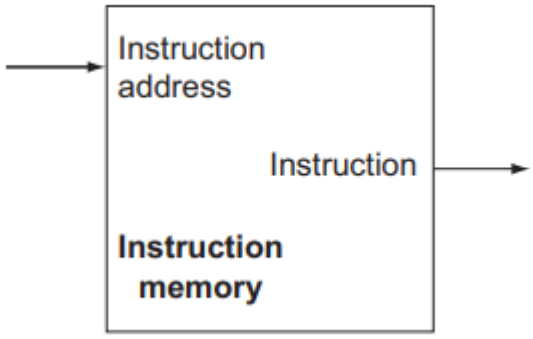
Memories are digital storage elements that help store a data and information in digital circuits. RAMs and ROMs are good examples of such memory elements. Storage elements can be modeled using one-dimensional arrays of type 'reg' and is called a memory. Each element in the memory may represent a word and is referenced using a single array index.



Instruction Memory: In RISC-V, the instruction memory is a component of the processor that holds the instructions to be executed by the processor. It is separate from the data memory, which holds data that is being processed by the instructions.

The instruction memory is typically implemented using a read-only memory (ROM) or flash memory, and it is organized as a linear array of memory locations, each containing one instruction. The instruction memory is accessed using the program counter (PC) register, which holds the address of the next instruction to be fetched.

When the processor needs to execute an instruction, it first fetches the instruction from the instruction memory by reading the memory location pointed to by the PC register. The instruction is then decoded and executed, and the PC is updated to point to the next instruction in memory.



CODE

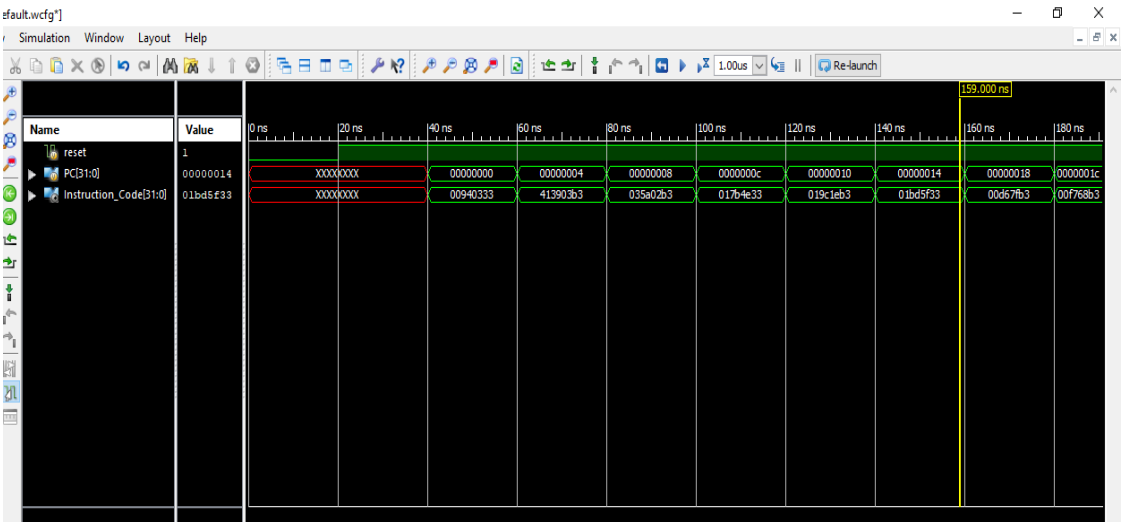
```
`timescale 1ns / 1ps
module INST_MEM(
   input [31:0] PC,
   input reset,
   output [31:0] Instruction_Code
);
   reg [7:0] Memory [31:0]; // Byte addressable memory with 32 locations

   // Under normal operation (reset = 0), we assign the instr. code, based on PC
   Assign Instruction_Code = {Memory[PC+3],Memory[PC+2],Memory[PC+1],Memory[PC]};

   // Initializing memory when reset is one
   always @(reset)
   begin
      if(reset == 1)
      begin
         // Setting 32-bit instruction: add t1, s0,s1
         Memory[3] = 8'h00;
         Memory[2] = 8'h94;
         Memory[1] = 8'h03;
         Memory[0] = 8'h33;
```

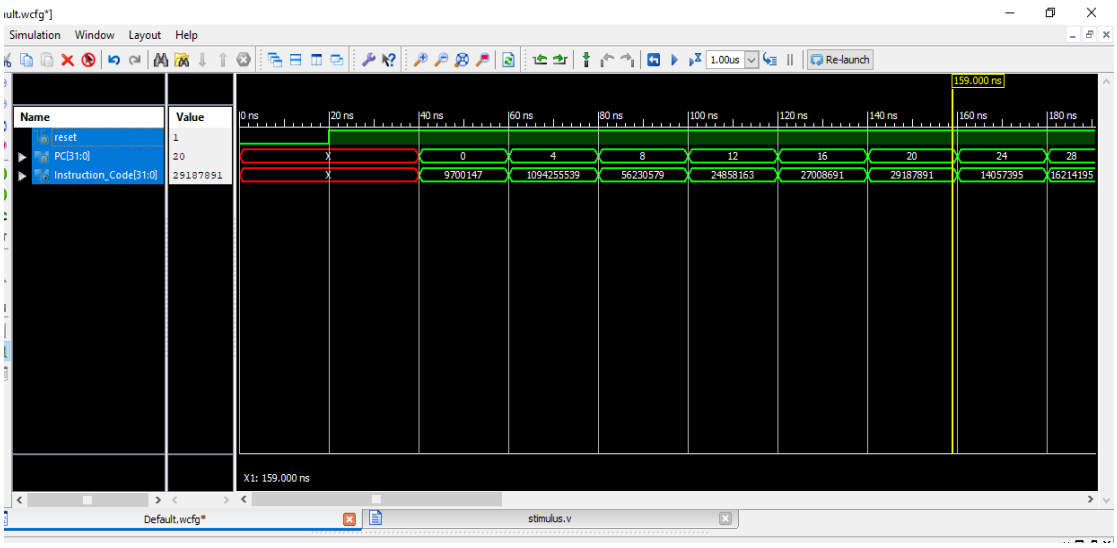
```
        // Setting 32-bit instruction: sub t2, s2, s3
        Memory[7] = 8'h41;
        Memory[6] = 8'h39;
        Memory[5] = 8'h03;
        Memory[4] = 8'hb3;

        // Setting 32-bit instruction: mul t0, s4, s5
        Memory[11] = 8'h03;
        Memory[10] = 8'h5a;
        Memory[9] = 8'h02;
        Memory[8] = 8'hb3;
        // Setting 32-bit instruction: xor t3, s6, s7
        Memory[15] = 8'h01;
        Memory[14] = 8'h7b;
        Memory[13] = 8'h4e;
        Memory[12] = 8'h33;
        // Setting 32-bit instruction: sll t4, s8, s9
        Memory[19] = 8'h01;
        Memory[18] = 8'h9c;
        Memory[17] = 8'h1e;
        Memory[16] = 8'hb3;
        // Setting 32-bit instruction: srl t5, s10, s11
        Memory[23] = 8'h01;
        Memory[22] = 8'hbd;
        Memory[21] = 8'h5f;
        Memory[20] = 8'h33;
        // Setting 32-bit instruction: and t6, a2, a3
        Memory[27] = 8'h00;
        Memory[26] = 8'hd6;
        Memory[25] = 8'h7f;
        Memory[24] = 8'hb3;
        // Setting 32-bit instruction: or a7, a4, a5
        Memory[31] = 8'h00;
        Memory[30] = 8'hf7;
        Memory[29] = 8'h68;
        Memory[28] = 8'hb3;
    end
  end

endmodule
```

Test Bench

```
`timescale 1ns / 1ps
module stimulus;

// Inputs
reg [31:0] PC;
reg reset;

// Outputs
wire [31:0] Instruction_Code;

// Instantiate the Unit Under Test (UUT)
INST_MEM uut (
    .PC(PC),
    .reset(reset),
    .Instruction_Code(Instruction_Code)
);

initial
   begin
      reset = 1'b0;
      #20 reset= 1'b1;
      #20 PC = 32'd0;
      #20 PC = 32'd4;
      #20 PC = 32'd8;
     #20 PC = 32'd12;
      #20 PC = 32'd16;
     #20 PC = 32'd20;
     #20 PC = 32'd24;
     #20 PC = 32'd28;

   end

   // Terminating after 150 clock cycles
   initial
   #250 $finish;

endmodule
```

OUTPUT Wave form (Hexadecimal and Binary form)





Data Memory: The Data Memory component is just an interface to the bus that interconnects the processor, main memory, and I/O devices. Since other devices use this bus, an enabled signal is required for both memory reads and memory writes.

Note that the Address input is shared by both the read port and the write port.

The input signal, Address, the output signal, RdData, and the control input signal RdEn make up the read port.

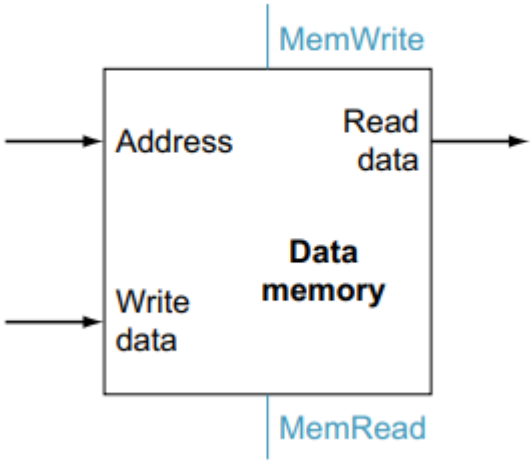
The input signal Address is a 32-bit signal that specifies a memory address.

The output signal RdData is a 32-bit signal. Its value is the data at the address specified by the Address input signal.

RdEn enables the read when its value is 1. The write does not happen if RdEn is 0. This signal is necessary to ensure the processor does not access the bus except when it needs to.

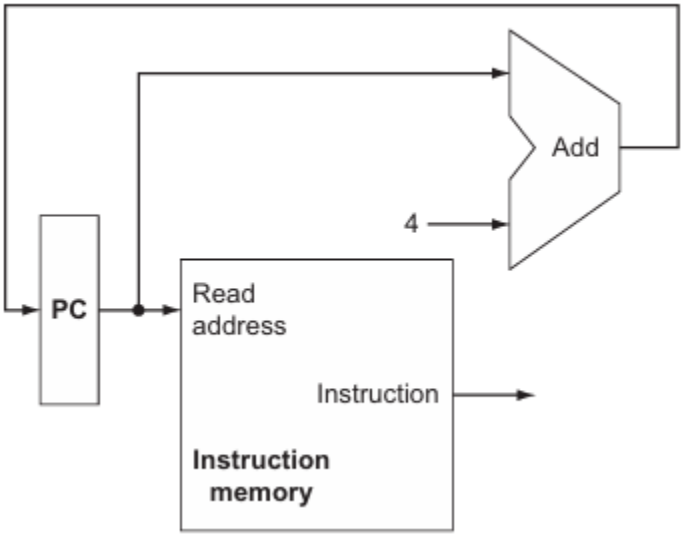
The input signals, Address and WrData, and the control input signal WrEn make up the write port.

The input signal Address is a 32-bit signal that specifies a memory address.
WrData specifies the data to be written at that address.



Lab Tasks:

1) Implement the following using code from last lab and instruction memory example.



2) Create a Data Memory having 32bit Write data input, 32bit address input and 32bit Read data output. The memory will also have 1bit MemWrite input, MemRead input and a clock. At positive edge of clock if MemWrite input is high, it will write the data and if MemRead is high, it will read the data.
   Create a test bench that will first write some data at some addresses and then read that data from those addresses.

Lab 6: Register File in Verilog

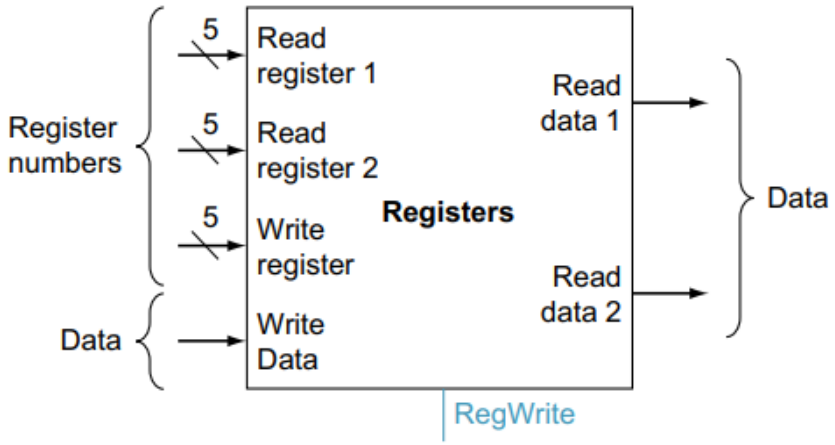
Lab Objectives:

- Develop Verilog coding proficiency by implementing a 32x32 register file.
- Understand the architecture of a register file, including registers, read/write operations, and control signals.
- Design a simple RAM in Verilog and test its functionality.
- Learn verification techniques by creating a test bench for comprehensive functional validation.
- Develop troubleshooting and debugging skills for identifying and addressing coding issues.

Register file: In RISC-V, the register file is a small, fast memory component that is used to hold the processor's general-purpose registers. The register file is typically implemented as a set of flip-flops or static random-access memory (SRAM) cells, which can be quickly read and written by the processor.

The RISC-V register file contains a set of 32 or 64-bit registers, depending on the implementation. The number of registers can vary depending on the implementation, but the standard RISC-V integer register file contains 32 registers labeled x0 to x31. Register x0 is hardwired to the value zero and cannot be written by any instruction.

The register file is used by the processor to hold intermediate results of computations, store function arguments and return values, and hold temporary values during program execution. Registers can be read by instructions that require their values, and they can also be written by instructions that produce new values.



INPUT PORTS:

ReadRegister1: 5-Bit address to select a register to be read through 32-Bit output port 'ReadRegister1'.

ReadRegister2: 5-Bit address to select a register to be read through 32-Bit output port 'ReadRegister2'.

WriteRegister: 5-Bit address to select a register to be written through 32-Bit input port 'WriteRegister'.

WriteData: 32-Bit write input port.

RegWrite: 1-Bit control input signal. (0: Read and 1:Write)

OUTPUT Ports:

ReadData1: 32-Bit registered output.

ReadData2: 32-Bit registered output.

Functionality:

'ReadRegister1' and 'ReadRegister2' are two 5-bit addresses to read two registers simultaneously. The two 32-bit data sets are available on ports 'ReadData1' and 'ReadData2', respectively. 'ReadData1' and 'ReadData2' are registered outputs. You can view it as if outputs of registers specified by ReadRegister1 and ReadRegister2 are written into output registers ReadData1 and ReadData2. 'RegWrite' signal is high during the rising edge of the clock if the input data is to be written into the register file. The contents of the register specified by address 'WriteRegister' in the register file are modified at the rising edge of the clock if 'RegWrite' signal is high.

Example: 32-bit register file having 32-bit data input, 1-bit clock, 1-bit reset, 1-bit enable input and 32-bit data output. The data of the output will be updated by input data on every positive edge of the clock only if the enable input is high. The data will reset on every positive edge of reset.

Code:

```
module register_file (
input [31:0] in,
output reg [31:0] out,
input clk, reset, en
);
```

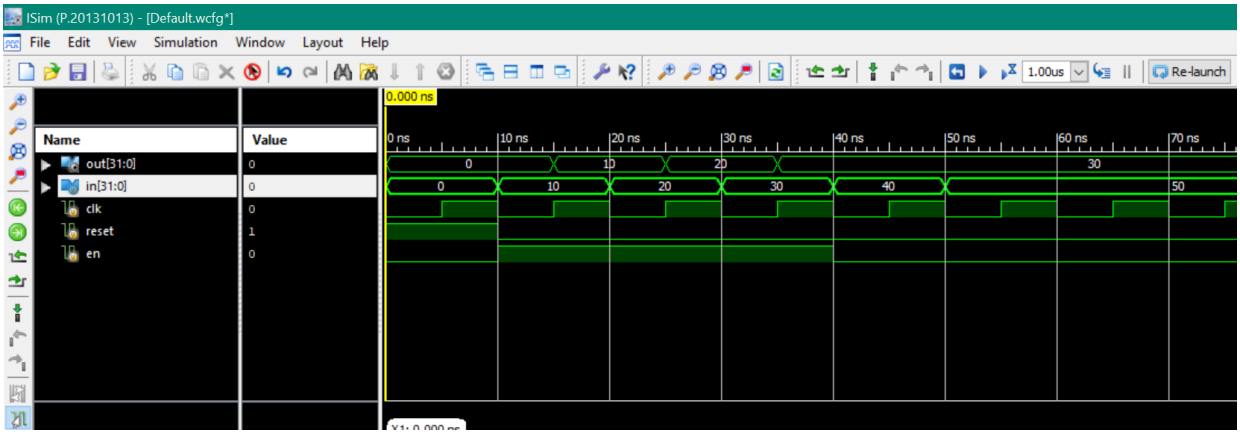
```
always @(posedge clk , posedge reset)
begin
    if (reset)
            out <= 32'h0;
    else if (en)
            out <= in;
end

endmodule
```

Test Bench:

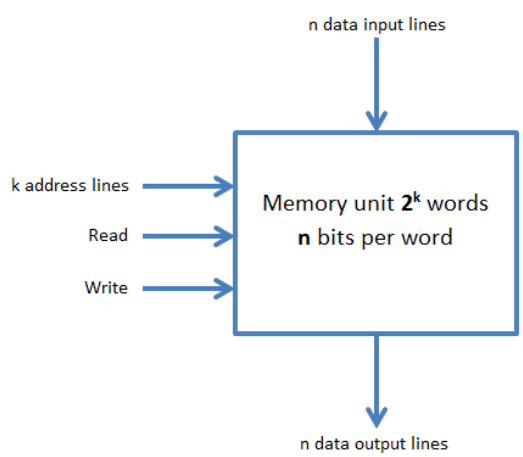
```
module test12;

    // Inputs
    reg [31:0] in;
    reg clk;
    reg reset;
    reg en;

    // Outputs
    wire [31:0] out;

    // Instantiate the Unit Under Test (UUT)
    register_file uut (
            .in(in),
            .out(out),
            .clk(clk),
            .reset(reset),
            .en(en)
    );

    always #5 clk = ~clk;

    initial begin
            // Initialize Inputs
            in = 0;
            clk = 0;
            reset = 1;
            en = 0;

            #10 en = 1; reset = 0; in = 10;
            #10 in = 20;
```

```
        #10 in = 30;
        #10 en = 0; in = 40;
        #10 in = 50;
        #10 in = 50;
        #100 $finish;

    end

endmodule
```

Waveform:



Example: Implementation of RAM:



The address input specifies the memory address to read from or write to, while n data input lines is the data to be written to the specified address. The write input enables writing to the specified address when it is high.

On every positive edge of the clk signal, the module checks whether write input is high, and if so, it writes the input value to the memory location specified by the address input. The module also outputs the value stored at the memory location specified by the address input on n data output lines.

In a RAM (Random Access Memory), the **write**, **read**, and **address** signals have the following functions:

1. **write**: This signal is used to write data to the memory. When **write** is high, the data provided on the input is written to the memory location specified by the **address** input.

2. **read**: This signal is used to read data from memory. When **read** is high, the data stored at the memory location specified by the **address** input is output.

3. **address**: This signal specifies the memory location to read from or write to. The number of bits required for the **address** signal depends on the size of the memory. For example, in a 16-bit RAM, the **address** signal would need 4 bits to address 16 memory locations.

Code

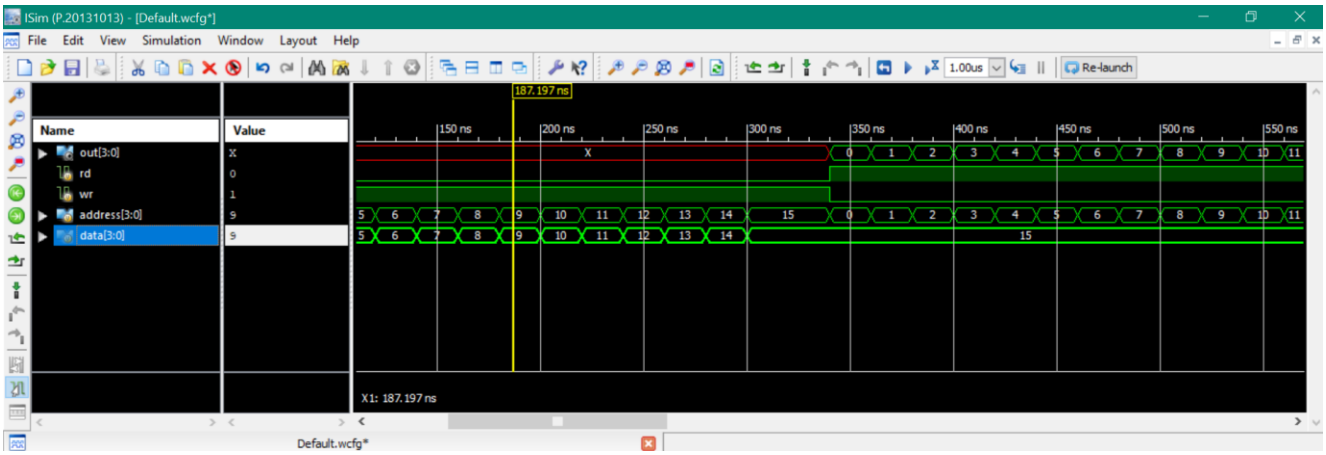
```
module ram_design(out,rd,wr,address,data);
input rd,wr;
input [3:0] data;
input [3:0] address;
output reg [3:0] out;
reg [3:0] model[15:0];

always @ (address)
begin
    if (wr)
        begin
            model[address] = data;
        end
    if (rd)
        begin
            out = model[address];
        end
end
endmodule
```

Test Bench

```
`timescale 1ns / 1ps
module ram_TB;

// Inputs
reg rd;
reg wr;
reg [3:0] address;
reg [3:0] data;

// Outputs
wire [3:0] out;

// Instantiate the Unit Under Test (UUT)
ram_design uut (.out(out), .rd(rd), .wr(wr), .address(address), .data(data));

initial
 begin
    // Initialize Inputs
        rd = 0; wr = 1; address = 4'b0000; data = 4'b0000;
#20   rd = 0; wr = 1; address = 4'b0001; data = 4'b0001;
#20   rd = 0; wr = 1; address = 4'b0010; data = 4'b0010;
#20   rd = 0; wr = 1; address = 4'b0011; data = 4'b0011;
#20   rd = 0; wr = 1; address = 4'b0100; data = 4'b0100;
#20   rd = 0; wr = 1; address = 4'b0101; data = 4'b0101;
#20   rd = 0; wr = 1; address = 4'b0110; data = 4'b0110;
#20   rd = 0; wr = 1; address = 4'b0111; data = 4'b0111;
#20   rd = 0; wr = 1; address = 4'b1000; data = 4'b1000;
#20   rd = 0; wr = 1; address = 4'b1001; data = 4'b1001;
#20   rd = 0; wr = 1; address = 4'b1010; data = 4'b1010;
#20   rd = 0; wr = 1; address = 4'b1011; data = 4'b1011;
#20   rd = 0; wr = 1; address = 4'b1100; data = 4'b1100;
#20   rd = 0; wr = 1; address = 4'b1101; data = 4'b1101;
#20   rd = 0; wr = 1; address = 4'b1110; data = 4'b1110;
#20   rd = 0; wr = 1; address = 4'b1111; data = 4'b1111;

#40;

        rd = 1; wr = 0; address = 4'b0000;
#20   rd = 1; wr = 0; address = 4'b0001;
#20   rd = 1; wr = 0; address = 4'b0010;
```

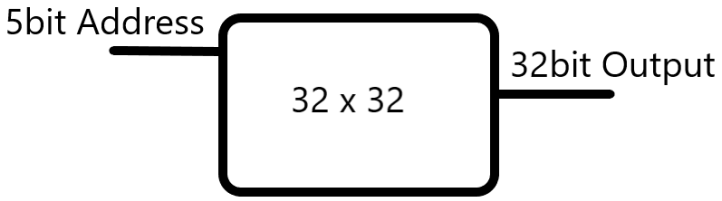
```
#20   rd = 1; wr = 0; address = 4'b0011;
#20   rd = 1; wr = 0; address = 4'b0100;
#20   rd = 1; wr = 0; address = 4'b0101;
#20   rd = 1; wr = 0; address = 4'b0110;
#20   rd = 1; wr = 0; address = 4'b0111;
#20   rd = 1; wr = 0; address = 4'b1000;
#20   rd = 1; wr = 0; address = 4'b1001;
#20   rd = 1; wr = 0; address = 4'b1010;
#20   rd = 1; wr = 0; address = 4'b1011;
#20   rd = 1; wr = 0; address = 4'b1100;
#20   rd = 1; wr = 0; address = 4'b1101;
#20   rd = 1; wr = 0; address = 4'b1110;
#20   rd = 1; wr = 0; address = 4'b1111;

end
endmodule
```

Wave form

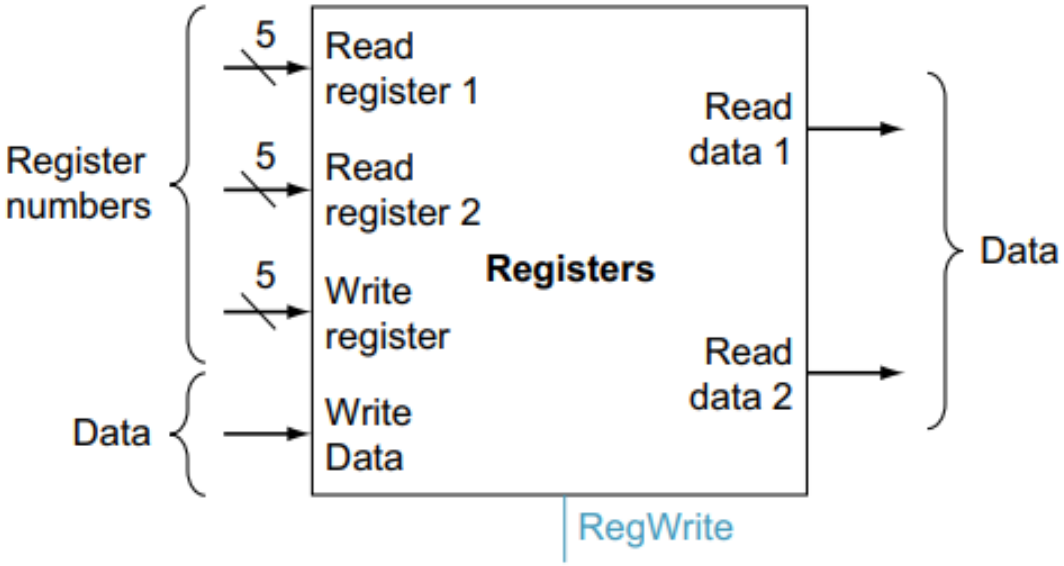


Lab Tasks:

1) Design a memory having 32 blocks and each block is of 32 bits. This will create a memory that is similar to 32 registers each of 32 bits. Write values from 0 to 31 to 0 to 31 blocks, e.g., register 0 will have value '0', register 1 will have value '1'. Make a 5bit address input that will read data from registers on every positive edge of clock and give a 32bit output.

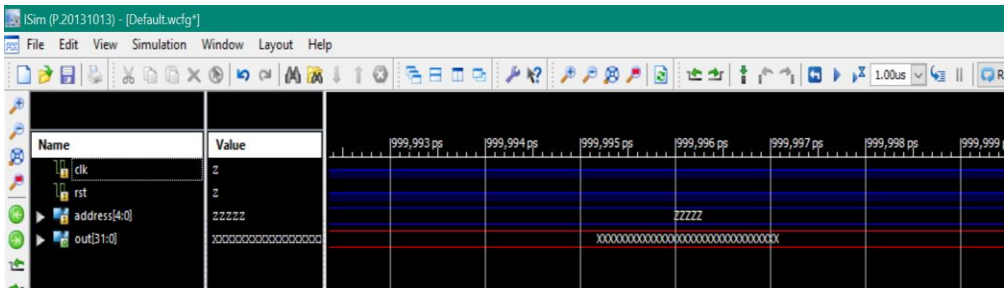


2) Design a register file with the specifications shown in the figure below. The write data and read data ports are 32bits. The data(register values) will be updated on every positive edge of the clock. A reset input is used to reset every register value to zero.



Common Errors / Instructions:

- Always create a new project for each task, this will save you from many software issues/errors.
- If you update inputs or outputs in an existing code, always create a new test bench file.
- If you encounter any issue and create a new project or test bench, don't copy paste whole old data, it can cause errors. Only update relevant data.
- Sometimes, in simulation, you may see blue and red lines, it can occur if you select main file (instead of test bench file) for simulation.



- If you think your code is fine and the test bench is not giving results, create a new test bench file and simulate it as it is without any modifications in the data. It will let you know if there was a problem with the test bench or your main code.
- If you're doing behavioral modeling, always use 'reg' with outputs, else, it will cause the 'reg' type error.

```
Line 25: Procedural assignment to a non-register out is not permitted, left-hand side should be reg/integer/time/genvar
Line 25: Procedural assignment to a non-register out is not permitted, left-hand side should be reg/integer/time/genvar
Line 3: Module <RegisterFile> ignored due to previous errors.
```

- If you use reg with any output and assign a value to it using 'assign', it will cause the following error.

```
28: Target <out> of concurrent assignment or output port connection should be a net type.
3: Module <RegisterFile> ignored due to previous errors.
ors
```

Lab 7: Data Path, Control Units, and Immediate Generation

Lab Objectives:

- Design Verilog modules for data path, control unit, and immediate generation.
- Implement data path elements.
- Generate control signals for instruction decoding and operation management.
- Develop immediate generation logic.
- Integrate modules for RISC-V processor design.

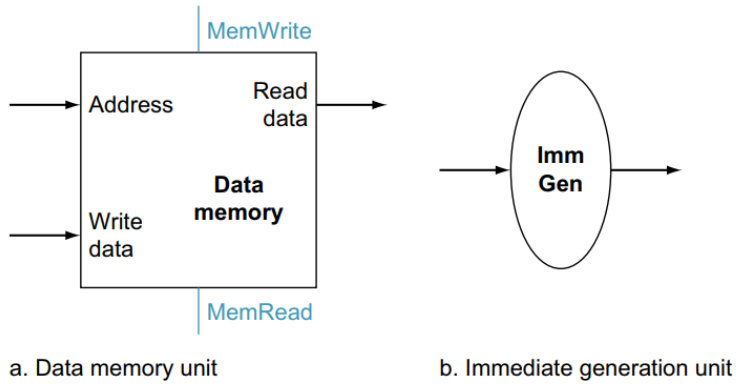
Data Path:

In modern computer architectures, the datapath plays a crucial role in executing instructions efficiently. A datapath comprises various components, including the register file, arithmetic logic unit (ALU), memory unit, and control unit, working together to process instructions. By integrating these components and control signals, you will gain practical insights into processor design and operation.

This unified datapath is designed to execute all instructions within a single clock cycle, ensuring that no datapath resource is utilized more than once per instruction. Hence, any component required multiple times must be duplicated.

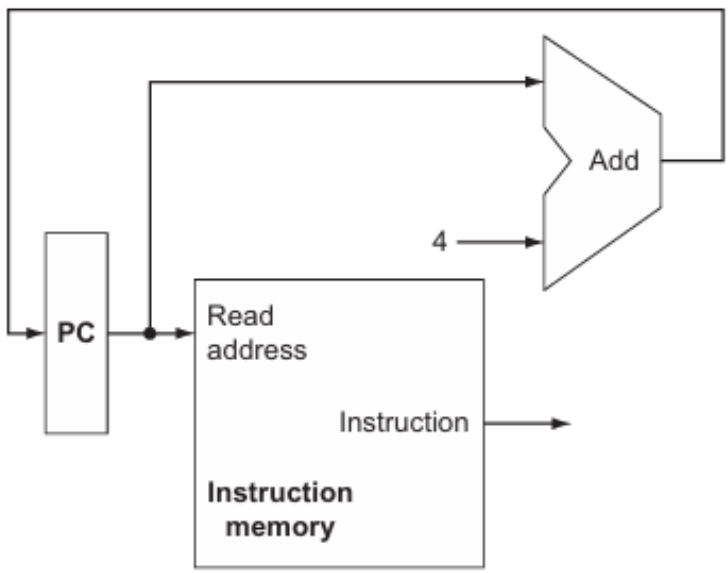
To support this unified architecture, we require separate memory units for instructions and data. While some functional units need duplication, many elements can be shared across different instruction flows.

For instance, in Figure 1, two additional units are introduced to implement load and store instructions. These include the data memory unit and the immediate generation unit. The data memory unit acts as a state element with inputs for address and write data, featuring separate read and write controls. Conversely, the immediate generation unit (ImmGen) processes 32-bit instructions, selecting a 12-bit field for load, store, and branch instructions, sign-extending it into a 32-bit output.



Example:

Combining the PC, Instruction Memory, and Adder in a single unit.



Code:

```
`timescale 1ns / 1ps

module task1 (
input clk, reset,
input [3:0] B,
output [31:0] out
  );

wire [31:0] pc_wire;
wire [31:0] outin;
```

```
PC32 PC (clk, reset, outin, pc_wire);
Adder4 Adder (pc_wire, B, outin);
INST_MEM Mem (pc_wire, reset, out);

endmodule


module Adder4 (
   input [31:0] A,
   input [3:0] B,
   output reg [31:0] Sum
);

   always @ (A or B) begin
      Sum <= A + B;
   end

endmodule

module PC32 (
   input clk,
   input reset,
   input [31:0] PC_in,
 output reg [31:0] PC_out
);

   always @ (posedge clk or posedge reset) begin
      if (reset) begin
         PC_out <= 32'b0;
      end else begin
         PC_out <= PC_in;
      end
   end

endmodule

module INST_MEM (
   input [31:0] PC,
   input reset,
   output [31:0] Instruction_Code
);
   reg [7:0] Memory [31:0]; // Byte addressable memory with 32 locations
```

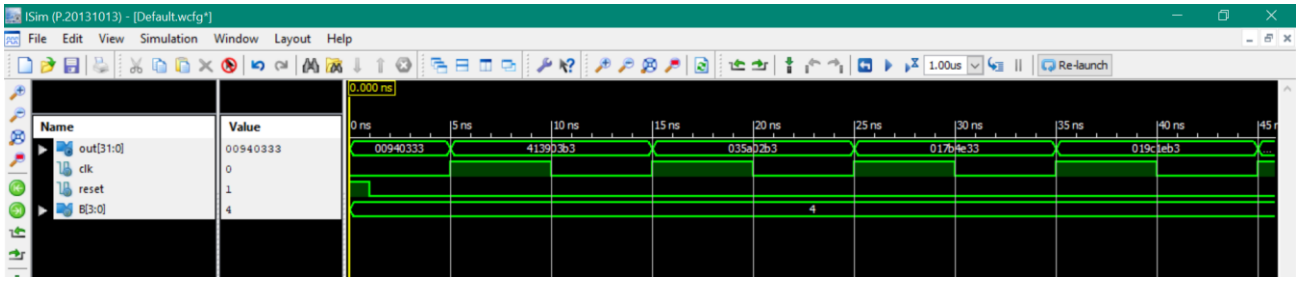
```
assign Instruction_Code =
{Memory[PC+3],Memory[PC+2],Memory[PC+1],Memory[PC]};

   // Initializing memory when reset is one
   always @(reset)
   begin
      if(reset == 1)
      begin
         // Setting 32-bit instruction: add t1, s0,s1
         Memory[3] = 8'h00;
         Memory[2] = 8'h94;
         Memory[1] = 8'h03;
         Memory[0] = 8'h33;
         // Setting 32-bit instruction: sub t2, s2, s3
         Memory[7] = 8'h41;
         Memory[6] = 8'h39;
         Memory[5] = 8'h03;
         Memory[4] = 8'hb3;

         // Setting 32-bit instruction: mul t0, s4, s5
         Memory[11] = 8'h03;
         Memory[10] = 8'h5a;
         Memory[9] = 8'h02;
         Memory[8] = 8'hb3;
         // Setting 32-bit instruction: xor t3, s6, s7
         Memory[15] = 8'h01;
         Memory[14] = 8'h7b;
         Memory[13] = 8'h4e;
         Memory[12] = 8'h33;
         // Setting 32-bit instruction: sll t4, s8, s9
         Memory[19] = 8'h01;
         Memory[18] = 8'h9c;
         Memory[17] = 8'h1e;
         Memory[16] = 8'hb3;
         // Setting 32-bit instruction: srl t5, s10, s11
         Memory[23] = 8'h01;
         Memory[22] = 8'hbd;
         Memory[21] = 8'h5f;
         Memory[20] = 8'h33;
         // Setting 32-bit instruction: and t6, a2, a3
         Memory[27] = 8'h00;
         Memory[26] = 8'hd6;
         Memory[25] = 8'h7f;
         Memory[24] = 8'hb3;
```

```
        // Setting 32-bit instruction: or a7, a4, a5
        Memory[31] = 8'h00;
        Memory[30] = 8'hf7;
        Memory[29] = 8'h68;
        Memory[28] = 8'hb3;
      end
    end

endmodule
```

TestBench:

```
module test123;

// Inputs
reg clk;
reg reset;
reg [3:0] B;

// Outputs
wire [31:0] out;

// Instantiate the Unit Under Test (UUT)
task1 uut (
        .clk(clk),
        .reset(reset),
        .B(B),
        .out(out)
);

always #5 clk = ~clk;

initial begin
        // Initialize Inputs
        clk = 0;
        reset = 1;
        B = 4;

        #1 reset = 0;
        // Wait 100 ns for global reset to finish
        #100 $finish;

        // Add stimulus here
```

```
end

endmodule
```



Immediate Generation:

Immediate generation in RISC-V refers to the process of creating immediate values that are encoded within certain instructions. Immediate values are constant values that are part of an instruction itself, rather than being stored in a register or memory location. These immediate values are typically used for various purposes such as specifying offsets, constants for arithmetic or logical operations, or providing small immediate data directly within an instruction.

In RISC-V, immediate values are often sign-extended and then inserted into specific bit fields within the instruction word. The immediate generation process involves generating the correct immediate value and placing it into the appropriate position within the instruction encoding.

Name (Bit position)	Fields 31:25	24:20	19:15	14:12	11:7	6:0
(a) R-type	funct7	rs2	rs1	funct3	rd	opcode
(b) I-type	immediate[11:0]		rs1	funct3	rd	opcode
(c) S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
(d) SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

Code:

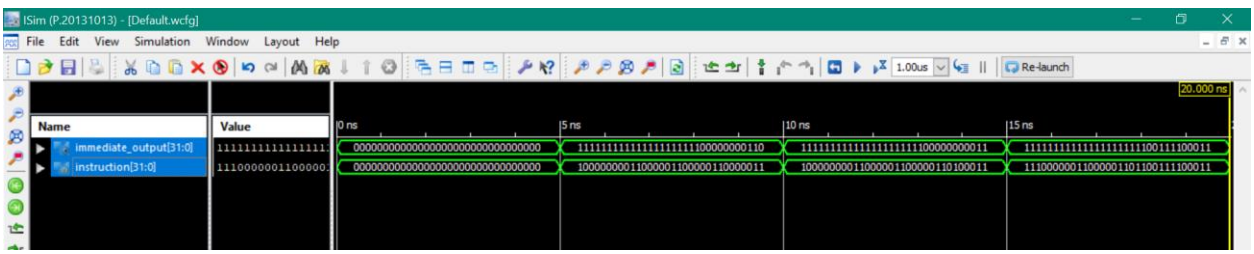
```
`timescale 1ns / 1ps

module imm_gen (
```

```
    input [31:0] instruction,
    output reg [31:0] immediate_output
);

parameter [6:0] LOAD_OPCODE = 7'b0000011;
parameter [6:0] STORE_OPCODE = 7'b0100011;
parameter [6:0] BRANCH_OPCODE = 7'b1100011;

wire [6:0] opcode = instruction[6:0];

wire [11:0] load_im = instruction[31:20];
wire [11:0] store_im = {instruction[31:25],instruction[11:7]};
wire [12:1] branch_im = {instruction[31], instruction[7], instruction[30:25],
instruction[11:8]};

always @(*) begin
    case(opcode)
        LOAD_OPCODE: immediate_output = { {20{load_im[11]}} , load_im};
        STORE_OPCODE: immediate_output = { {20{store_im[11]}} , store_im};
        BRANCH_OPCODE: immediate_output = 2*{{20{branch_im[12]}} ,
branch_im}; // left shift
        default: immediate_output = 32'b0;
    endcase
end

endmodule
```

Test Bench:

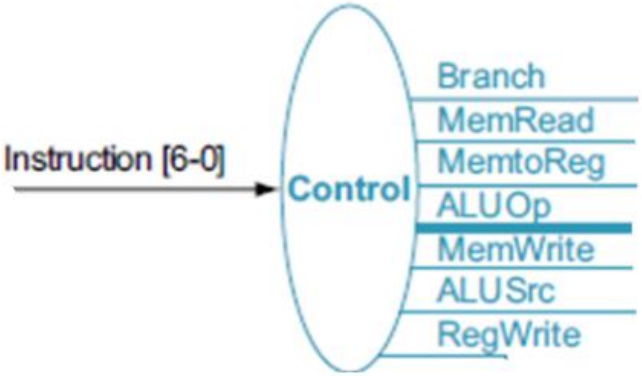
```
`timescale 1ns / 1ps

module imm_gen_test;

        // Inputs
        reg [31:0] instruction;

        // Outputs
        wire [31:0] immediate_output;

        // Instantiate the Unit Under Test (UUT)
        imm_gen uut (
                .instruction(instruction),
                .immediate_output(immediate_output)
        );
```

```
        initial begin
                // Initialize Inputs
                instruction = 0;

                #5 instruction = 32'b10000000011000001100000110000011;
                #5 instruction = 32'b10000000011000001100000110100011;
                #5 instruction = 32'b11100000011000001101100111100011;

                // Wait 100 ns for global reset to finish
                #5 $finish;

                // Add stimulus here

        end

endmodule
```



Control Unit:

The control unit in RISC-V, as in other architectures, is responsible for coordinating and controlling the operation of the CPU. It plays a crucial role in executing instructions by managing the fetch-decode-execute cycle.



Instruction	ALUSrc	Memto-Reg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R-format	0	0	1	0	0	0	1	0
lw	1	1	1	1	0	0	0	0
sw	1	X	0	0	1	0	0	0
beq	0	X	0	0	0	1	0	1

The first row of the table corresponds to the R-format instructions (add, sub, and, and or). For all these instructions, the source register fields are rs1 and rs2, and the destination register field is rd; An R-type instruction writes a register (RegWrite = 1), but neither reads nor writes data memory. When the Branch control signal is 0, the PC is unconditionally replaced with PC + 4; otherwise, the PC is replaced by the branch target if the Zero output of the ALU is also high. The ALUOp field for R-type instructions is set to (10) to indicate that the ALU control should be generated from the funct fields. The second and third rows of this table give the control signal settings for lw and sw. These ALUSrc and ALUOp fields are set to perform the address calculation. MemRead and MemWrite are set to perform memory access. Finally, RegWrite is set for a load to cause the result to be stored in the rd register. The ALUOp field for branch is set for subtract (ALU control = 01), which is used to test for equality. MemtoReg in the last two rows of the table is replaced with X for don't care. This type of don't care must be added by the designer, since it depends on knowledge of how the data path works.

Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

The control function for the simple single-cycle implementation is completely specified by the above truth table.

ALU Control Unit:

In RISC-V, the ALU (Arithmetic Logic Unit) control unit is responsible for generating control signals that determine the operation to be performed by the ALU based on the instruction being executed. The ALU control unit interprets the opcode and other relevant fields of the instruction to produce the appropriate control signals for the ALU.

Input: The ALU control unit takes as input the opcode field of the instruction, along with any other necessary fields that determine the operation to be performed by the ALU.

Operation Decoding: Based on the opcode and other relevant fields of the instruction, the ALU control unit decodes the operation to be performed by the ALU. This typically involves mapping opcode values to specific ALU operations.

Control Signal Generation: Once the operation is decoded, the ALU control unit generates control signals that specify the ALU operation to be executed. These control signals may include signals to select the ALU operation (e.g., addition, subtraction, etc.), as well as any additional signals needed to configure the ALU for the desired operation.

Output: The control signals generated by the ALU control unit are then used to configure the ALU to perform the specified operation on the input operands.

Lab Tasks:

Task 1: Implement the following Control Unit in Verilog.

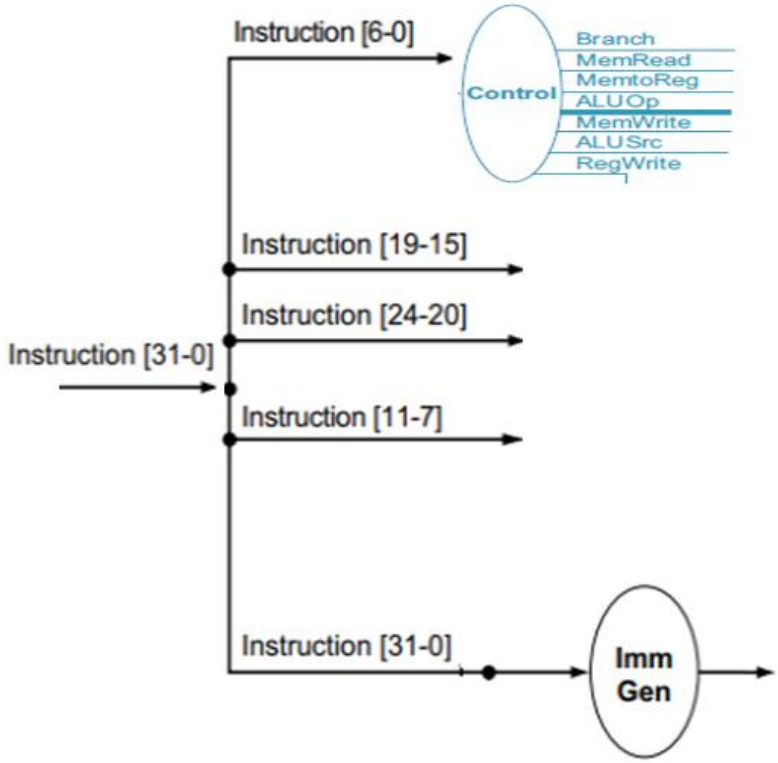
Input or output	Signal name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Task 2: Implement the following ALU Control Unit in Verilog.

ALUOp, Funct7, and Funct3 are inputs and Operation is the 4-bit output.

ALUOp		Funct7 field	Funct3 field			
ALUOp1	ALUOp0	I[30]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	0010
X	1	X	X	X	X	0110
1	X	0	0	0	0	0010
1	X	1	0	0	0	0110
1	X	0	1	1	1	0000
1	X	0	1	1	0	0001

Extra Task: Decode the 32-bit instructions from instruction memory into the following outputs .



Lab 8-11: Implementation of RISC-V Processor in Verilog

Lab Objectives:

- Understand RISC-V Architecture, including instruction formats, opcode decoding, and register operations.
- Understand the working of RISC V Architecture.
- Learn single-cycle processor design methodology.
- Implement instruction fetch, decode, execute, and write-back stages in Verilog.
- Develop skills in test bench development and simulation for verification.

Introduction:

RISC-V is an open-source instruction set architecture (ISA) that is revolutionizing the way we design and implement computer processors. Developed at the University of California, Berkeley, RISC-V is a modern, modular, and extensible ISA that is designed to be simple, efficient, and scalable.

History of RISC-V:

RISC-V was first developed in 2010 by Yunsup Lee, Andrew Waterman, and David Patterson at the University of California, Berkeley. The initial version, RISC-V v1, was a 32-bit ISA that was designed to be a simple and efficient alternative to existing ISAs. Since then, RISC-V has evolved to include 64-bit and 128-bit versions, as well as various extensions and modifications.

Key Features of RISC-V:

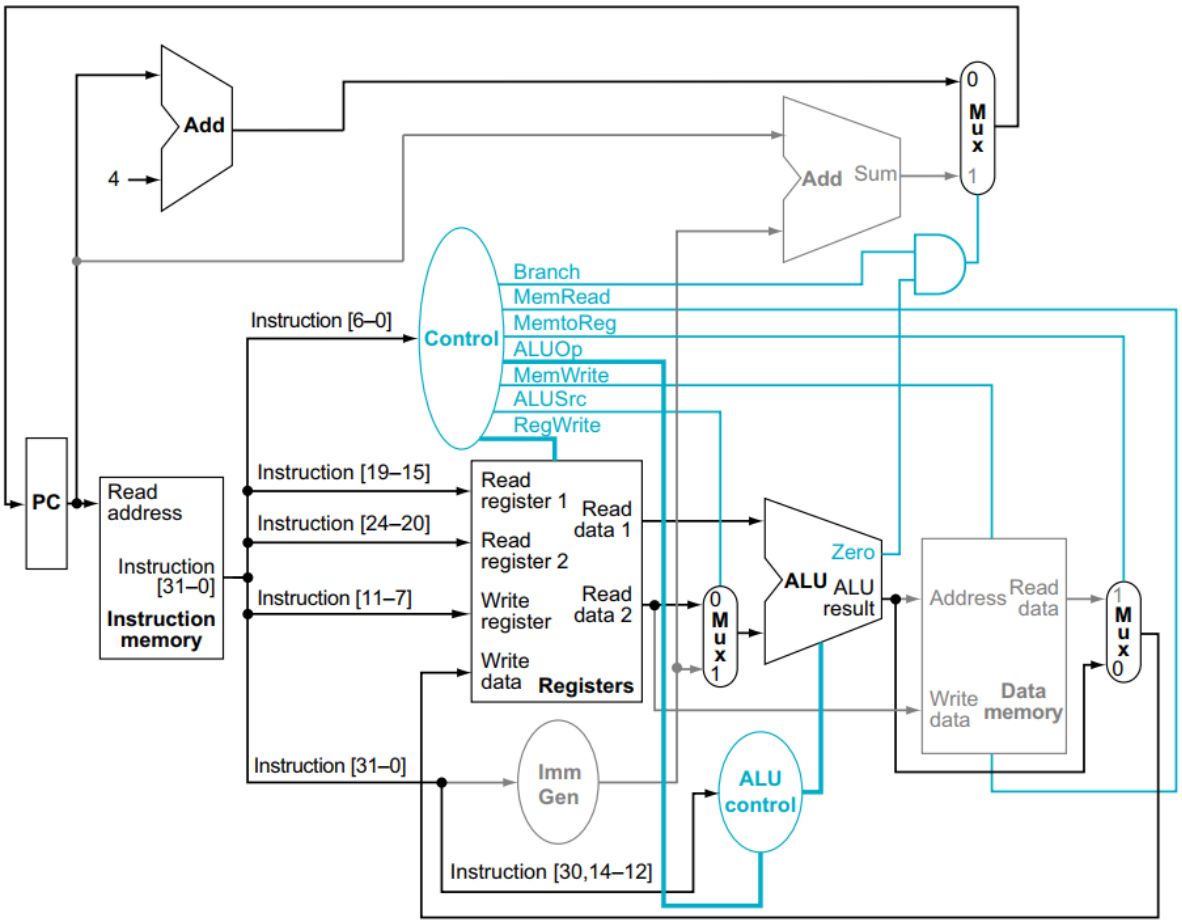
RISC-V has several key features that make it an attractive choice for processor design:

- Simple and Modular: RISC-V has a simple and modular design, with a small number of instructions and a regular instruction format.
- Extensible: RISC-V is designed to be extensible, with a large number of reserved instruction codes and a modular design that allows for easy addition of new instructions and features.
- Open-Source: RISC-V is an open-source ISA, which means that anyone can use, modify, and distribute it without restriction.
- Free: RISC-V is free to use, with no licensing fees or royalties.
- High-Performance: RISC-V is designed to be high-performance, with a focus on efficiency and scalability.

Advantages of RISC-V:

RISC-V has several advantages over other ISAs:

- Customizability: RISC-V's modular design and open-source nature make it easy to customize and modify for specific applications.
- Portability: RISC-V is designed to be portable, with a focus on software compatibility across different implementations.
- Security: RISC-V has a number of security features, including hardware support for secure boot and secure execution.



The simple datapath for the core RISC-V architecture combines the elements required by different instruction classes

Main Components:

All the main components in the above figure are already created in previous labs.
Here's a brief introduction of each component.

<u>Program Counter (PC):</u>

- The Program Counter (PC) is a special-purpose register that holds the address of the next instruction to be fetched from memory.

- It is automatically incremented after each instruction fetch operation.
- In a RISC-V processor, the PC is essential for maintaining the instruction execution sequence and determining the memory address of the next instruction to fetch.

Adder:

- An adder is a digital circuit responsible for performing addition operations on binary numbers.
- In a RISC-V processor, adders are commonly used for various purposes such as incrementing the PC to fetch the next instruction, computing memory addresses for load and store operations, and performing arithmetic calculations within the ALU.
- They can be simple ripple-carry adders or more complex carry-lookahead or carry-save adders, depending on the performance requirements of the processor.

Multiplexers (MUXes):

- Multiplexers, often abbreviated as MUXes, are digital circuits that select one of several input signals and route it to the output based on control signals.
- In a RISC-V processor, MUXes are used to select between multiple sources of data or control signals, depending on the current instruction being executed or the processor's state.
- They play a crucial role in datapath design by enabling dynamic routing of data and control signals to various functional units such as the ALU, register file, and memory units.

Arithmetic Logic Unit (ALU):

- The Arithmetic Logic Unit (ALU) is a digital circuit responsible for performing arithmetic and logical operations on binary data.
- In a RISC-V processor, the ALU executes operations specified by instructions fetched from memory, such as addition, subtraction, AND, OR, and XOR.
- It operates on data stored in registers or memory and produces results based on the instruction operands.
- The ALU's efficiency and speed significantly impact the overall performance of the processor, making it a crucial component in datapath design.

Register File:

- The register file stores temporary data and operands for arithmetic and logic operations.
- It typically consists of a set of registers accessible by the processor's execution units.
- RISC-V processors often have a small number of general-purpose registers for frequently used data.

- These registers are directly accessible by the instruction set architecture (ISA) and play a crucial role in instruction execution.
- The register file facilitates fast access to operands, reducing memory access overhead and enhancing performance.

Instruction Memory:

- The instruction memory is a storage unit that holds the program instructions to be executed by the processor.
- It stores the binary representation of instructions fetched from the program stored in main memory or cache.
- In a RISC-V processor, the instruction memory is typically implemented using static random-access memory (SRAM) or read-only memory (ROM) technologies.
- Instructions are fetched from the instruction memory sequentially based on the address provided by the program counter (PC).
- The instruction memory is a critical component in the instruction fetch stage of the processor's pipeline, providing instructions to the instruction decode stage for further processing.
- Its speed and efficiency directly impact the overall performance of the processor, making it essential for efficient instruction execution.

Immediate Generation Unit:

- The immediate generation unit generates immediate values used as operands in instructions.
- It extracts immediate fields from instruction words and sign-extends them to the required width.
- In RISC-V processors, the immediate generation unit plays a crucial role in processing instructions with immediate operands.
- It ensures that immediate values are correctly formatted and ready for use in arithmetic, logical, or branching operations.
- The immediate generation unit enhances instruction execution efficiency by providing immediate values directly to the ALU or other functional units.

Control Unit:

- Coordinates the flow of data and activities within the processor.
- Generates control signals based on instruction opcodes and processor state.
- Manages instruction fetch, decode, execute, and writeback stages.
- Activates specific functional units such as ALU and memory units as required.
- Ensures proper instruction execution according to the RISC-V ISA.
- Vital for maintaining overall processor operation and efficiency.

ALU Control Unit:

- Determines the ALU operation based on the instruction opcode.
- Interprets instruction opcodes to generate appropriate control signals.
- Specifies ALU operations for addition, subtraction, logical AND, OR, XOR, etc.
- Ensures the ALU operates correctly according to the executed instruction.
- Coordinates ALU activities with the instruction execution process.
- Essential for enhancing the processor's ability to efficiently handle diverse computational tasks.

Data Memory:

- The data memory unit is responsible for storing and retrieving data from memory.
- It includes the data cache and memory management logic necessary for accessing memory locations.
- In RISC-V processors, the data memory unit interacts with the memory hierarchy to access main memory or cache levels efficiently.
- It handles load and store operations, transferring data between the processor and memory.
- The data memory unit's performance influences memory access latency and overall system throughput.

Lab Task:

Design and implement a single-cycle RISC-V processor in Verilog, using the examples and code from past labs as a guide.

Here are the steps you can follow to implement the RISC-V Processor.

Step 1: Design the Instruction Memory

- Define the instruction memory size
- Design the instruction memory module
- Write Verilog code for instruction memory

Step 2: Design the Data Memory

- Define the data memory size
- Design the data memory module
- Write Verilog code for data memory

Step 3: Design the Program Counter

- Define the program counter width (e.g. 32 bits)
- Design the program counter module
- Write Verilog code for program counter

Step 4: Design the Adders

- Define the adder width (e.g. 32 bits)
- Design the adder module
- Write Verilog code for adder

Step 5: Design the Decoder

- Define the instruction formats (R-type, I-type, S-type, SB-type)
- Design the decoder module
- Write Verilog code for decoder

Step 6: Design the Immediate Generation

- Define the immediate width (e.g. 12 bits)
- Design the immediate generation module
- Write Verilog code for immediate generation

Step 7: Design the ALU Control Unit

- Define the ALU control signals (e.g. opcode, func3, func7)
- Design the ALU control unit module
- Write Verilog code for ALU control unit

Step 8: Implement the 7 Instructions

- Implement the R-type instructions (ADD, SUB, AND, OR)
- Implement the I-type instruction (LW)
- Implement the S-type instruction (SW)
- Implement the SB-type instruction (BEQ)

Step 9: Integrate the Components

- Integrate the instruction memory, data memory, program counter, adders, decoder, immediate generation, ALU control unit, and instructions
- Write Verilog code for the integrated processor

Step 10: Write the Testbench

- Write a testbench to simulate and test the processor
- Test the processor with sample instructions and data

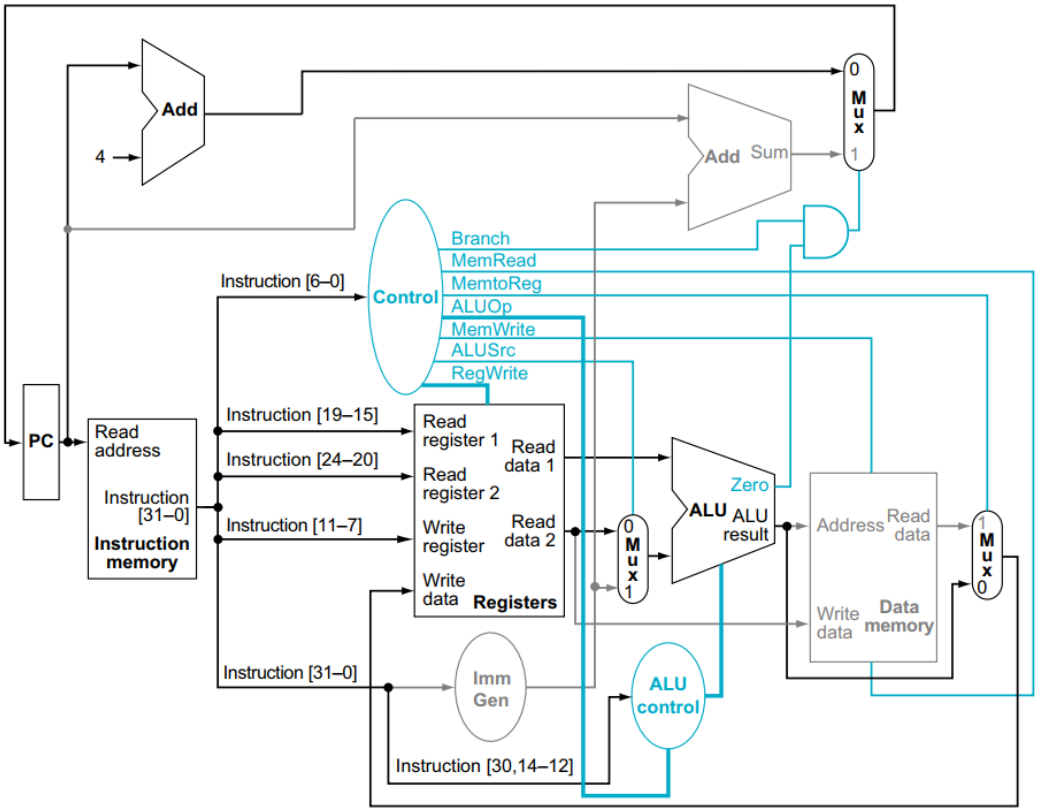
Step 11: Simulate and Test

- Simulate the processor using a Verilog simulator
- Test the processor with different instructions and data

Step 12: Debug and Optimize

- Debug any errors or issues found during simulation and testing
- Optimize the processor for performance and area

Step 13: Finalize

- Finalize the Verilog code for the RISC-V single cycle processor

The implementation of the RISC-V processor will be a multi-session project, spanning 3-4 lab sessions.